AI 芯片工程研习室

离线学习手册·正文、练习与完整答案

资料核对截至 2026-11-08

目录

1 阅读说明 2

2 学习目标与路线 3

3 录屏归纳索引 7

4 先把问题写成可验收的规格 10

5 SFU 与 LUT：分清数学误差和硬件一致性 15

6 XLS 与流水线：从功能图到带时序的实现 18

7 验证闭环：怎样证明 AI 写的实现符合规格 21

8 架构模型：先界定问题，再选择抽象层级 25

9 算子融合与 L1：数据搬运为什么支配结果 29

10 模型验证与校准：把“看起来合理”变成可检查 32

11 PPA 与候选筛选：证据、约束和 Pareto 35

12 工程 Agent：状态、上下文与可验证交付 38

13 选择 AI 芯片工程 Flow：按任务、证据与成本比较 43

14 综合项目：交付一个经得起复查的工程闭环 48

15 工程 Flow 比较 52

16 实验模型与边界 56

17 术语索引 58

18 完整来源索引 60

1 阅读说明

从算法到 RTL，从算子图到架构性能：让每一步都有可核验的证据

面向已有计算机体系结构、数字逻辑基础的读者。三天建立主线，两周完成一个可复现实验；核心是规格、数值、验证、性能建模与受约束的 AI flow。公开版脱敏归纳录屏，不发布原视频、人物信息或内部源码。

本手册把规格、功能、性能与 AI 工作流连接成一条可核验的证据链。建议先读示范，再补全步骤，最后独立完成变式；每道开放练习都附完整参考答案和自评量表。数值算例与录屏主张分开标注，工具能力与已执行证据分开说明。

这是离线静态版本。纸面推演、公式、题目、答案与来源均完整保留；需要运行代码的实验，应在本地下载公开示例后执行。外部来源及验证报告链接需要网络，PDF 内部目录和章节链接可直接跳转。

课程网站：https://andyzr-coder.github.io/ai-workflow-learning-lab/

公开示例源码：https://github.com/andyzr-coder/ai-workflow-learning-lab

2 学习目标与路线

最终能力包括解释机制、画数据与控制路径、定量估算、调试反例、比较方案，以及设计独立验证。

2.1 问题与能力对应

1. 怎样把模糊目标变成可验收合同?

层次：必须掌握

先修：体系结构/数字逻辑基础

模块：先把问题写成可验收的规格

练习与实验：画图题：画出 4 core 共用一个外部内存通道的数据路径，并标注可相加和不可相加的速率。；补全步骤：64 GiB 数据在假设的持续有效速率 16 GB/s 下，忽略启动与协议开销，传输时间估计是多少?；独立设计：为 AI 生成 exp 数据通路列出至少六项必须确认的验收条件。

来源：[video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳） ·[src-anthropic-patterns] Building effective agents

2. 数学误差和 RTL 错误如何分离?

层次：必须掌握

先修：函数、浮点与误差

模块：SFU 与 LUT：分清数学误差和硬件一致性

练习与实验：补全计算：sin 的 N=32、F=10、无饱和条件下，总误差上界是多少?；反例题：为什么把输出 F 从 10 增加到 14，精度可能下降?；独立设计：为 FP32 sigmoid 列一个包括均匀、特殊和边界输入的验证计划。；实验：LUT 精度：分段、位宽与饱和

来源：[video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳） ·[src-xls-float] XLS Floating-point routines

3. 流水级、II 和背压怎样影响性能?

层次：必须掌握

先修：依赖与时序逻辑

模块：XLS 与流水线：从功能图到带时序的实现

练习与实验：补全计算：L=6、II=2、f=250 MHz，首结果延迟与稳态吞吐是多少?；画时序题：画出 3 级流水线，下游在边沿 3、4 不 ready 时，前 4 笔事务的状态。；迁移题：若乘法器每 3 拍才能接受一个输入，如何扩展目前每级 1 拍的模型?；实验：流水线与背压：看见每拍资源占用

来源：[video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳） ·[src-xls-overview] XLS: Accelerated HW Synthesis - Overview ·[src-xls-quickstart] XLS Tools Quick Start ·[src-xls-dslx] XLS DSLX Reference

4. 测试全绿为什么仍可能错误?

层次：必须掌握

先修：位宽、握手、复位

模块：验证闭环：怎样证明 AI 写的实现符合规格

练习与实验：补全步骤：补全饱和加法参考模型，并给出两个溢出反例。；调试题：out_valid=1、out_ready=0 时，AI 生成的模块仍更新 data。画两拍状态并解释错误。；独立设计：设计一个 16 位带符号饱和乘法的验证计划。不能复用本例的无符号阈值。；实验：独立参考：为什么全绿也可能全错

来源：[video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳） ·[src-cocotb-2] cocotb 2.0.0 Documentation ·[src-cocotb-timing] The Timing Model - cocotb 2.0.0 ·[src-verilator-overview] Verilator Overview

5. 何时受算力、带宽、容量或依赖限制?

层次：必须掌握

先修：吞吐、带宽、单位

模块：架构模型：先界定问题，再选择抽象层级

练习与实验：补全计算：保持默认模型，仅把峰值计算速率从 128 提升到 256 Gop/s，理想重叠时间如何变?；调试题：模型把每核带宽乘以核数后性能特别好，实际没有提升。给出三个排查点。；陌生场景：为一个多请求共享压缩引擎建立最小事件模型，并声明不能回答的问题。；实验：架构瓶颈：算力、带宽与重叠

来源：[video] 用户提供的 2026-11-08 研讨录屏 (脱敏归纳) ·[src-roofline] Roofline: An Insightful Visual Performance Model for Multicore Architectures

6. 融合何时省流量，何时因容量失效?

层次：必须掌握

先修：算子图与内存层级

模块：算子融合与 L1：数据搬运为什么支配结果

练习与实验：补全计算：上述 1,048,576 元素改用 FP32，Y 容量与写读流量分别是多少?；画图题：画 X/Y/Z 的活跃区间，解释 4MiB 容量何时能容纳三个各 2MiB 张量。；设计题：为带 mask 的 softmax 提出融合方案和一个不等价反例。

来源：[video] 用户提供的 2026-11-08 研讨录屏 (脱敏归纳) ·[src-roofline] Roofline: An Insightful Visual Performance Model for Multicore Architectures ·[src-mlir-transform] MLIR Transform Tutorial - Chapter 0

7. 如何校准并检查新形状?

层次：深入选学

先修：模型、微基准与误差

模块：模型验证与校准：把 “看起来合理” 变成可检查

练习与实验：补全计算：若保留集 9MiB 实际为 1.40ms，而预测 1.00ms，相对参考值的偏差是多少?；反例题：模型漏掉输出写回但调低带宽后拟合通过。如何揭露?；设计题：为数据相关 TopK 模型制定校准与保留集方案。

来源：[video] 用户提供的 2026-11-08 研讨录屏 (脱敏归纳) ·[src-roofline] Roofline: An Insightful Visual Performance Model for Multicore Architectures ·[src-timeloop-overview] Timeloop/Accelergy Overview ·[src-accelsim] Accel-Sim author framework repository

8. 如何公平比较与筛可行候选?

层次：必须掌握

先修：RTL、验证与 STA

模块：PPA 与候选筛选：证据、约束和 Pareto

练习与实验：补全计算：用 44.70 与 98.97mW 计算相对下降比例，分母选谁?；反例题：候选 A 比 B 功耗低 20%，但 A 是 100MHz、B 是 200MHz。如何重做比较?；独立设计：给 5 轮 AI 候选制定停止规则与交付目录。；实验：PPA 筛选：可行域与 Pareto 前沿

来源：[video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳） ·[src-orfs-flow] OpenROAD Flow Scripts - RTL-to-GDSII Flow ·[src-orfs-tutorial] OpenROAD Flow Scripts Tutorial

9. 如何建立有限、可恢复的 AI 工具闭环?

层次：必须掌握

先修：独立参考和状态机

模块：工程 Agent：状态、上下文与可验证交付

练习与实验：示范：示范：饱和加法器对 200+100 输出 44。把一次修复写成可验收记录。；补全：补全：作业提交回执丢失，应进入 ____ 状态，并通过 ____ 查询。若生成 5 s、编译 5 s、两项独立检查 20/30 s、汇总 2 s，理想并行总耗时为 ____。；独立设计：为 AI 辅助的双资源事件模拟器设计协作流程：一个 agent 写调度，另一个写验证。如何避免双方把同一个错误重叠假设写进实现和测试?

来源：[src-anthropic-patterns] Building effective agents ·[src-langgraph-retry] LangGraph Fault tolerance ·[src-autochip-2025] Automatically Improving LLM-based Verilog Generation using EDA Tool Feedback ·[src-cocotb-2] cocotb 2.0.0 Documentation

10. 哪些流程适合当前任务，证据与成本是什么?

层次：深入选学

先修：AI 闭环与架构模型

模块：选择 AI 芯片工程 Flow：按任务、证据与成本比较

练习与实验：示范：示范：AI 写的事件模型把共享读写端口当成两个独立端口，如何用最小实验发现?；补全：补全：同一 30 任务，A 成功 18 个共 90,000 token；B 成功 24 个共 150,000 token。求成功率、百分点差和每成功任务总 token，并指出尚不能得出的结论。；独立设计：为一个未知架构的矩阵计算 + 激活融合项目设计两条可比较 Flow：一条快速分析，一条带事件队列的模型。怎样证明后者值得额外成本?

来源：[src-timeloop-overview] Timeloop/Accelergy Overview ·[src-timeloop-v4] Timeloop+Accelergy v4 Architecture ·[src-mlir-transform] MLIR Transform Tutorial - Chapter 0 ·[src-maestro] MAESTRO Documentation

11. 如何独立交付可复现的证据包?

层次：必须掌握

先修：验证、校准与流程选择

模块：综合项目：交付一个经得起复查的工程闭环

练习与实验：补全步骤：饱和加法变成回绕加法后，预计什么反例会失败?；调试题：模型在三个形状上很准，如何判断能否用于新架构?；独立项目：交付 SFU 或融合算子的证据包，并按六项量表自评。

来源：[video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳） ·[src-verilator-overview] Verilator Overview ·[src-cocotb-2] cocotb 2.0.0 Documentation ·[src-openai-repair] OpenAI - Build iterative repair loops with Codex

2.2 三天建立主线·每天约 2-3 小时，选读核心段落

第 1 天：从规格到正确性

先把问题写成可验收的规格 ·SFU 与 LUT：分清数学误差和硬件一致性 ·验证闭环：怎样证明 AI 写的实现符合规格

能力产出：手算误差；独立完成饱和加法反例；写一份验收合同。

第 2 天：从局部到系统

XLS 与流水线：从功能图到带时序的实现 ·架构模型：先界定问题，再选择抽象层级 ·算子融合与 L1：数据搬运为什么支配结果

能力产出：画流水线时序与数据流；完成 Roofline 与融合搬运算例。

第 3 天：证据驱动的 AI flow

PPA 与候选筛选：证据、约束和 Pareto ·工程 Agent：状态、上下文与可验证交付 ·选择 AI 芯片工程 Flow：按任务、证据与成本比较

能力产出：比较同任务流程；选择一条最小可复现路线；自评陌生场景题。

2.3 两周深入·每天约 60-90 分钟

第 1-3 天：建立词汇与合同

先把问题写成可验收的规格 ·SFU 与 LUT：分清数学误差和硬件一致性 ·XLS 与流水线：从功能图到带时序的实现

能力产出：整理术语，完成示范与补全题；熟悉内容可以先答题后跳过。

第 4-5 天：验证实现

验证闭环：怎样证明 AI 写的实现符合规格

能力产出：运行 RTL 案例、保留 PASS 和故意注入错误的 FAIL 证据。

第 6-8 天：建模与融合

架构模型：先界定问题，再选择抽象层级 ·算子融合与 L1：数据搬运为什么支配结果

能力产出：画事件与资源占用；统一单位与层级；做参数敏感度。

第 9-10 天：校准与 PPA

模型验证与校准：把 “看起来合理” 变成可检查 ·PPA 与候选筛选：证据、约束和 Pareto

能力产出：设计微基准和保留集；对候选做可行域与 Pareto 筛选。

第 11-12 天：搭建 AI 工作流

工程 Agent：状态、上下文与可验证交付 ·选择 AI 芯片工程 Flow：按任务、证据与成本比较

能力产出：冻结参考、建立工具合同、设置重试与预算；比较失败恢复。

第 13-14 天：综合项目

综合项目：交付一个经得起复查的工程闭环

能力产出：提交可复现仓库、证据包与未完成项；用量表自评并安排复习。

2.4 复习记录

建议在 1、3、7、14 天后复习，可按需要调整。先凭记忆解释机制，再完成一个变式；若不能独立完成，缩短复习间隔。在纸面学习日志中分开记录阅读、练习证据和自评结果。

3 录屏归纳索引

本节是匿名技术归纳，不包含原始音视频、人物信息、原始逐字稿或内部源码。时间码相对录屏起点；录制从介绍中途开始。自动摘要不作为新的发言或决议。

3.1 00:00:00-00:04:47 从功能参考到 RTL

先建立功能参考，再经过中间表示、调度和代码生成，最后进行验证与实现评估。不同阶段需要用同一组输入比较结果。先确认功能一致，再讨论性能优化；抽样测试不等于完整等价证明。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

XLS 与流水线：从功能图到带时序的实现

3.2 00:04:47-00:09:29 优化闭环与人工判断

示例通过实现结果发现问题，再由人工提出优化要求。讨论强调目标、工具结果和工程判断需要相互校验。区分已演示的步骤、人工参与和仍在规划的自动化能力。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

PPA 与候选筛选：证据、约束和 Pareto

3.3 00:09:29-00:16:43 模块目标从系统需求而来

讨论从局部实现转向系统性能：模块指标如何分配，抽象模型如何与实际硬件连接，以及速度与精度如何取舍。模型必须回答明确的设计问题，并说明能够可信回答到哪一层。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

先把问题写成可验收的规格

3.4 00:16:43-00:20:09 建立可配置的架构模型

用计算单元、存储层级、容量、带宽和数据流描述硬件。即使省略计算单元内部细节，也不能省略关键数据搬运。先画清计算与存储之间的数据路径，再为资源赋值。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

架构模型：先界定问题，再选择抽象层级

3.5 00:20:09-00:23:39 容量不足如何放大带宽瓶颈

局部存储放不下工作集会增加外部搬运，多核共享链路又会加剧竞争。讨论同时暴露了旧配置与新规格不同步的问题。容量、带宽、并发和规格版本必须一起检查。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

架构模型：先界定问题，再选择抽象层级

3.6 00:23:39-00:30:35 图分解、分块与算子融合

模型前端构建计算图，硬件后端应用资源与数据流约束。融合和分块策略会改变中间数据驻留位置及搬运量。不考虑可实现的融合策略，可能高估实际所需的数据搬运。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

算子融合与 L1：数据搬运为什么支配结果

3.7 00:30:35-00:37:31 评估既定架构，还是搜索新架构

讨论区分两类任务：在已有资源下预测性能，以及反过来选择指令、资源配比与模块划分。能效还涉及有用工作与无效翻转。先写清搜索变量、约束和评价目标，避免把评估器当成完整设计器。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

架构模型：先界定问题，再选择抽象层级

3.8 00:37:31-00:40:10 逻辑相同，物理代价未必相同

计算与存储的距离、存储块组织和布线会改变同一逻辑操作的成本。高层分析需要来自实际实现的反馈。逻辑数据量相同，不代表物理能耗和时序成本相同。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

PPA 与候选筛选：证据、约束和 Pareto

3.9 00:40:10-00:50:11 用具体问题定义模型边界

参与者要求以具体算子和数据路径验证模型价值，并追问周期数、带宽等参数从哪里来。粗模型与详细模拟器需要不同证据。为每个参数标明假设、规格或测量来源，并为具体问题选择合适工具。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

模型验证与校准：把“看起来合理”变成可检查

3.10 00:50:11-00:54:44 复核配置与分析报告

讨论检查规格版本、容量、链路带宽、计算能力和缓冲策略，并要求报告暴露对设计决策有用的指标。旧配置的结果不能替代当前设计的评估。

核对状态：中高：发言与展示结构相互支持；不采用未核准的内部数值。

模型验证与校准：把“看起来合理”变成可检查

3.11 00:54:44-01:01:39 吞吐模型仍需拆解复杂算子

建议以吞吐、带宽和局部容量建立快速成本模型；复杂算子仍需拆成基本操作。固定指令周期与访存重叠假设需要说明，不能默认为精确硬件行为。

核对状态：中高：讨论清晰；这是建模建议，非已完成能力证明。

算子融合与 L1：数据搬运为什么支配结果

3.12 01:01:39-01:04:57 为什么执行时间会变化

访存竞争、端口冲突、形状和数据排列会影响观察到的执行时间。固定实现的算术执行成本与等待资源的时间需要分开。把内在计算时间、数据服务时间和竞争等待分别建模。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

架构模型：先界定问题，再选择抽象层级

3.13 01:04:57-01:11:39 理想性能参考与现实偏差

容量峰值依赖既定分块方案，理想时间依赖带宽和重叠假设。讨论要求考虑竞争、气泡并通过测量校准尚未确认的吞吐。理论参考用于定位改进方向，不能直接当作硬件实测承诺。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

模型验证与校准：把“看起来合理”变成可检查

3.14 01:11:39-01:14:48 让分析产出可执行的下一步

将重复结构归并成少量代表性用例，输出逐算子的理想成本与限制资源。依赖访问模式的冲突交给更详细的模型。好的报告要告诉设计者哪里值得优化，以及何时需要更细的验证。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

综合项目：交付一个经得起复查的工程闭环

3.15 01:14:48-01:17:33 批量与多芯片扩展

批量变化会改变中间状态和存储需求；扩展到多芯片后，还要纳入任务划分、互联带宽和通信竞争。扩展计算资源时，必须同时扩展通信与存储模型。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

架构模型：先界定问题，再选择抽象层级

3.16 01:17:33-01:19:28 用可执行模型支撑结论

详细通道效率需要具体用例和细粒度模拟。数值结论应由可复现的模型代码产生，AI可辅助建模与整理。让模型代码和验证证据承担数值结论。

核对状态：中高：基于两遍本地转写与画面交叉复核；术语和数字仍需谨慎。

工程 Agent：状态、上下文与可验证交付

3.17 01:19:28-01:23:45 会后摘要画面

主要技术发言已经结束，后段停留在会议摘要等桌面画面。自动生成的摘要不作为新的发言或决议。区分原始发言、自动摘要与后续课程整理。

核对状态：高：片段用途已由画面与全程语音检测确认；不把自动摘要当作新发言。

选择 AI 芯片工程 Flow：按任务、证据与成本比较

4 先把问题写成可验收的规格

模块 1 ·必须掌握·建议 45 分钟

整段讨论反复追问的不是 AI 能不能写代码，而是工程师有没有定义正确问题、参数和验收条件。规格是人机协作的接口，也决定最后能证明什么。

先修：无额外先修

横向关联：模型验证与校准：把 “看起来合理” 变成可检查 ·工程 Agent：状态、上下文与可验证交付

4.1 能力目标

- 把自然语言目标转换成输入、输出、约束和证据
- 区分视频陈述、工具报告、实测与教学估算
- 为一个性能指标声明统计边界和单位
- 画出从需求到候选交付的证据链

4.2 为什么需要：同一句 “更快”，可能代表不同目标

降低一笔操作的首结果延迟、提高连续输入吞吐、降低整层模型耗时，是三种目标。把 “实现 sin，尽可能快” 交给 AI，会把输入范围、误差、特殊值、接口、工艺与资源预算留成隐含选择。工程师必须先定义可讨论、可拒绝的验收合同。

用熟悉的 ISA 类比：一条指令需要规定可见行为，微架构才有优化空间。但 AI 的自然语言需求没有 ISA 那样严格，不能假定它会自动补全唯一正确的解释。

4.3 整段录屏归纳：结论、争论与后续计划

覆盖整段 83 分 45 秒录屏。主要技术发言至约 79 分 28 秒，后段为摘要等桌面画面与短音频事件。录制从一段介绍中途开始，不能代替未录入的会议开头。

以两遍本地语音转写为主，结合画面复核；自动生成的侧栏纪要只作辅助。公开内容已去除人员、内部型号、内部配置数值和原始逐字稿。

已展示或讲述的能力

1. **分阶段工具链**：介绍了从功能参考、中间表示、调度到 RTL 与实现评估的链路，以及跨阶段的抽样结果比较。(00:00-04:39)
   边界：抽样比较不等于全输入空间的形式化等价证明。
2. **有人工参与的优化迭代**：展示了查看实现结果、由人工提出改进要求，再执行优化的过程。(04:47-09:29)
   边界：不能据此声称已实现完全自主闭环；示例改善也不能外推到所有设计。
3. **可配置的架构分析框架**：展示了硬件配置、算子分解、融合方案和性能报告的组织方式。(16:43-30:35；50:11-54:44)
   边界：示例中存在规格更新，数值需要重新计算与校准。
4. **代码执行数值分析**：明确说明数值运行由模型代码完成，AI用于辅助工作。(78:41-79:28)
   边界：执行了代码不自动保证假设正确，仍需独立验证。

必须保留的争论

1. **正向评估与架构搜索**：评估一套已给定资源，与选择 ISA、资源比例及模块划分，是不同任务；后者仍是扩展方向。(30:35-37:31)
2. **快速模型与详细模拟器**：快速模型应服务早期资源配置和软件切分；端口冲突、调度和详细通道效率需要更细模型。两者的边界与参数依赖仍需说清。(40:10-49:56；54:44-64:47)
3. **固定周期还是吞吐估算**：固定算术的内在执行成本与实际等待时间不同；访存竞争、形状及数据组织会影响观测延迟。(61:39-64:47)

4. **逻辑模型与物理成本**：相同的逻辑计算和数据搬运可能有不同物理成本；高层模型应接受实现反馈，而非宣称直接预测底层能耗。(37:31-40:10)
5. **理想性能与现实性能**：理想带宽、重叠和利用率假设决定模型边界。理论参考可以帮助定位方向，不能当作实测保证。(64:57-71:34)

下一步归纳计划

1. **建立参数来源与版本台账**：统一当前规格，标注参数是经验假设、设计约束还是测量值，并保留变更记录。(20:09-23:39；43:24-49:56)
   产物：可版本化配置与参数来源表
2. **用小用例回答当前设计问题**：选取排序筛选、向量处理、局部带宽和非连续搬运后矩阵计算等代表性场景，先明确要回答的问题。(40:10-43:17)
   产物：带输入形状和预期瓶颈的微基准清单
3. **完善算子分解与融合成本**：对复杂算子拆解基本操作；比较分块与融合方案，说明容量约束及数据重叠假设。(54:44-61:39；71:39-74:47)
   产物：逐算子理想成本、限制资源和融合候选报告
4. **校准关键吞吐和时间估算**：对尚未核准的计算能力进行测量，将模型与实现对照，解释竞争、气泡和数据布局造成的偏差。(47:06-49:56；57:01-57:49；64:57-71:34)
   产物：假设、测量条件、误差与适用范围记录
5. **加入批量及互联扩展**：支持批量变化，并把多芯片任务划分、互联带宽和通信竞争纳入可配置分析。(74:48-77:33)
   产物：匿名教学拓扑上的批量与通信扫描实验
6. **建立从粗到细的验证流程**：用快速模型筛选方案，再把端口冲突、调度及具体通道效率问题交给详细模型。(71:39-74:47；77:33-78:41)
   产物：模型选择条件与升级验证检查单

未获证明的事项

- 未证明完全自动的端到端芯片设计或 PPA 优化闭环已经完成。
- 未证明抽样回归覆盖全部功能输入。
- 未证明示例中的架构预测等同实测硬件性能。
- 未证明粗模型已经精确建模全部指令周期、存储冲突或物理能耗。
- 未证明讨论中提出的所有后续方向都已形成最终承诺或完成实现。

课程中的匿名架构、公式推导和实验数值属于教学延伸，应与录屏中的主张分别标注。

4.4 系统位置：三条相互约束的路径

需求 / 负载 / 精度 → 功能规格 → 参考模型 → 候选实现
硬件配置 / 时序 / 工艺 → 资源模型 → 工具运行 → 指标
测试 / 不变量 / 校准 → 证据检查 → 取舍决策 → 交付

功能路径决定计算什么；资源路径决定哪里慢、哪里花面积和能量；证据路径决定哪种结论被允许。录屏里“给抽象 spec，让 AI 探索”与“人先参与微结构”的分歧，可以转化为探索空间与验收深度的选择，而不是非此即彼。

4.5 对象与状态：一份最小合同

```
task_id: sat-add-demo
inputs: unsigned a[7:0], b[7:0]
function: min(255, a+b)
protocol: valid/ready; stalled output remains stable
reset: active-low asynchronous; output valid clears
acceptance: exhaustive arithmetic + stalls + reset
budget: bounded repair attempts; keep every failing seed
artifacts: source hash, tool versions, log, verdict
```

模型任务另加 shape、dtype、MAC 口径、内存层级、全芯片/每 core、峰值/持续带宽、数据分布和负载阶段。空白表示未知，不自动表示零，更不能由 AI 随意补值。

4.6 证据等级：什么能支持什么结论

- 规范要求：定义必须满足的行为；需要版本与条款。
- 工具或厂商实现：说明某版本怎么工作；不自动代表所有实现。
- 实测：记录设备、输入、测量边界、采样和误差。
- 仿真：记录模型、参数、仿真器与假设；不能写成硅后实测。
- 解析估算：快速寻找瓶颈、上下界和敏感度；精度需要校准。
- 视频陈述：保留时间戳和上下文；原报告未提供的部分标为未独立复现。
- 路线图和推断：未来计划或基于假设的判断；不能写成已完成能力。

录屏侧栏本身是 AI 会议摘要。它可能把 PPA 写成 PVA，或者误抄百分比。课程核对原讲述、幻灯片、公式与一手资料，不把侧栏当作事实权威。

4.7 定量例题：一个参数表如何错出四倍

教学系统有 4 个 core，外部共享总带宽 32 GB/s，有效利用率 0.5。全芯片有效带宽为 16 GB/s，不是 4×32×0.5=64 GB/s。后者只有在每个 core 真有独立 32 GB/s 通道、且上游不共享瓶颈时才成立。搬运 64 GB 分别会估算为 4 s 与 1 s，误差四倍。

检查顺序：先确认层级与共享关系，再确认单位，最后引入效率。1 GB=10^9 byte，1 GiB=2^{30} byte；1 GHz=10^9 cycle/s。如果某参数是 word/cycle，还要乘以 word 的字节数。

4.8 取舍、误解与练习

早期规格应允许标记未知与给范围，避免为了齐全编造精确数值；进入验收时则必须消除会影响结论的歧义。文档越长不一定越好，关键是每个指标都能指向一个测试或报告。

练习先抄写上面的合同，再补全“带宽、精度与统计范围”，最后独立为一个 softmax 加速器定义合同。不要把方案偏好写成未证明的硬性事实。

4.9 能力训练与完整答案

练习 1 •画图题

题目：画出 4 core 共用一个外部内存通道的数据路径，并标注可相加和不可相加的速率。

补全步骤与提示：从 core→ 片上互连 → 共享通道 → 内存画起。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 每核本地端口能力可以汇总为需求，但外部总流量受共享 32 GB/s 峰值约束。有效 16 GB/s 是所选效率假设；不能每核重复计入。

自评量表

- 资源分层
- 共享通道
- 峰值有效值
- 单位和假设

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 ·补全步骤

题目：64 GiB 数据在假设的持续有效速率 16 GB/s 下，忽略启动与协议开销，传输时间估计是多少？

补全步骤与提示：先将 GiB 换成 byte，再除 byte/s。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 64×2^30/(16×10^9)=4.294967296 s。这是在固定有效速率假设下的估计；只有 16 GB/s 被定义为不能超过的带宽上限时，Q/B 才是相应严格下界。

自评量表

- 正确二进制容量
- 十进制带宽
- 计算结果
- 说明效率假设与下界条件

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 ·独立设计

题目：为 AI 生成 exp 数据通路列出至少六项必须确认的验收条件。

补全步骤与提示：思考数值、接口、硬件、测量四类。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 输入域与边界处理；dtype 和舍入/特殊值；误差口径和阈值；valid/ready 与复位；延迟和 II；工艺时钟面积功耗预算；独立参考、固定测试与报告。

自评量表

- 数值契约
- 时序接口
- 资源与统计边界
- 可执行证据

评分：满分 4 分；开放答案采用自评，不自动判定正确。

4.10 快速检查

1. 参数表写 “32 GB/s”，第一件该确认的是什么？

A. 直接乘 core 数
B. 是全芯片共享还是每核独占、读写口径是什么
C. 换成更大的数字
D. 把它叫实测带宽

答案：B：带宽的资源边界决定能否相加。单位正确仍可能因共享关系错误而产生倍数级误差。

4.11 来源与阅读边界

- [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）
- [src-anthropic-patterns] Building effective agents

5 SFU 与 LUT：分清数学误差和硬件一致性

模块 2 ·必须掌握·建议 70 分钟

视频以 exp、sin、sigmoid 的 SFU（Special Function Unit，特殊函数单元）为案例。重点不在记住某组系数，而在把“函数近似”和“逐位硬件语义”分成两条验证线。

先修：先把问题写成可验收的规格

横向关联：XLS 与流水线：从功能图到带时序的实现 ·PPA 与候选筛选：证据、约束和 Pareto

5.1 能力目标

- 画出区间变换、分段寻址、系数表、算术和特殊值路径
- 分别计算绝对误差、相对误差与采样最大值
- 解释 FMA 与分开乘加的舍入差别
- 用反例指出均匀采样与浮点精度声明的边界

5.2 为什么需要：把复杂函数变成可实现的数据路径

通用数学库可以使用复杂范围归约和迭代。专用硬件常用 LUT（Look-up Table，查找表）与低阶多项式减少运行时工作。表越细，近似通常越好，但系数存储、译码和访存代价也变大。

像 CPU 微码表一样，查表把一部分计算成本提前到离线；类比边界是数值近似还要承担误差预算，而微码通常实现精确定义的语义。

5.3 对象与数据路径

```
输入 x → 分类/范围归约 → 段号 i 与局部变量 t
                          ↓
                   系数表 a[i], b[i], c[i]
                          ↓
                   u = FMA(a[i],t,b[i])
                   y = FMA(u,t,c[i])
                          ↓
                  后处理 / 特殊值 / 输出
```

录屏约 03:00 的幻灯片使用二次多项式与两次 FMA，局部变量 t=x-origin[i]。系数由 Python 维护并输出 DSLX。Python 在这里承担模型和生成器角色，并不意味着任意 Python 程序能直接综合成电路。

5.4 算法与状态：四个数值对象不要混在一起

- 数学目标 f(x)：例如理想 sin。
- 实数近似 p(x)：拟合或插值造成第一部分误差。
- 有限精度行为模型 p_fp(x)：系数编码、中间运算和舍入造成第二部分误差。
- RTL 输出 p_rtl(x)：应按接口和时序与 p_fp 对齐，再检查逐位或规定容差。

FMA（Fused Multiply-Add，融合乘加）对 a×b+c 只在最终结果舍入；分开乘法与加法通常有两次舍入。数学上等价的表达式，有限精度下不一定逐位等价。XLS 浮点文档还列出具体操作的非规格化数、舍入和异常标志限制，不能只看“FP32”标签。XLS Floating-point routines，FMA 与各操作说明。

5.5 约束：误差口径与采样边界

绝对误差 e_abs=|y−f(x)|；相对误差 e_rel=|y−f(x)|/|f(x)|。f(x)=0 时后者未定义，接近零时也容易失真。可报告排除阈值后的相对误差，同时保留全域绝对误差。ULP 是相邻可表示值间距口径，需指定格式、数值区域与特殊值处理。

视频幻灯片给出 exp 的 [-20,0]、sin 的 [-π,π]、sigmoid 的 [-6,6] 各 2048 个均匀样本。它们是所测点的统计范围，不是所有 FP32 编码的穷举证明。必须额外检查段界两侧、尾端、接近零、溢出/下溢和特殊值。

5.6 完整例题：线性插值上界如何推导

配套教学模型刻意简化为线性插值，不复现视频的二次 FP32 电路。对 sin 在 [-π,π] 上均匀分 N 段，$h=2\pi/N$，且 $|f''|\le 1$。插值余项满足 $|f-L|\le h^2/8$。N=16 时，上界约 0.0192766；若输出小数位 F=10，$\Delta=2^{-10}$，最近舍入增加至多 $\Delta/2\approx 0.000488281$，总上界约 0.0197649。

上界假设节点与中间插值精确、没有输出饱和。实验用 JS 双精度逼近这种教学模型，4097 点给的是采样最大误差。分段数翻倍，插值项上界变为四分之一；量化项不会自动减小，因而会逐渐成为瓶颈。节点表代理存储量为 (N+1)×B bit；不含地址逻辑，不是硬件面积。

5.7 取舍与失败：位数越多也可能更糟

固定总位宽 B 而增加小数位 F，会提升分辨率但缩小可表示动态范围。exp(x) 可能发生饱和，最终误差反而增大。小数位不是越多越好，必须结合范围、误差目标与资源预算。

失败例：用 Python 默认双精度乘加对照目标 FP32 FMA，然后把差异全部当成 RTL bug。正确做法是先实现明确的有限精度参考，并把逼近误差与实现偏差分开定位。

5.8 练习：从示范到陌生函数

先手算 N=16、F=10 的上界，再到 “LUT 精度” 实验确认趋势。补全 N=32 的计算，最后尝试 exp 并把 B 降至 8、F 设为 7，解释为什么细分表格仍救不了饱和。最后为 tanh 提出范围归约与特殊值策略。

5.9 能力训练与完整答案

练习 1 •补全计算

题目：sin的 N=32、F=10、无饱和条件下，总误差上界是多少?

补全步骤与提示：$h=2\pi/32$；算 $h^2/8$ 后加 2^{-11}。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 约 0.00481914+0.000488281=0.00530742。它是模型假设下解析上界，不是本次采样实测值。

自评量表

- 代入段长
- 二阶导数上界
- 量化项
- 区分上界与采样

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 ·反例题

题目：为什么把输出 F 从 10 增加到 14，精度可能下降?

补全步骤与提示：固定 B=16，先计算最大正数。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 B=16,F=14的最大输出约 1.99994，exp(2)≈7.389 被限幅；范围损失可以远大于量化改善。

自评量表

- 固定总位宽
- 计算值域
- 构造超界输入
- 指出修复条件

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 ·独立设计

题目：为 FP32 sigmoid 列一个包括均匀、特殊和边界输入的验证计划。

补全步骤与提示：分数学误差与实现语义两个列表。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 均匀与高曲率区加密；每段边界两侧；零、极小值、正负极值、NaN/∞；参考明确 FMA、舍入与次正规数策略；记录最坏输入、分位数、绝对/相对误差边界。

自评量表

- 双验证线
- 边界与特殊值
- 有限精度语义
- 可复现报告

评分：满分 4 分；开放答案采用自评，不自动判定正确。

5.10 快速检查

1. 2048 个样本与 RTL 一致，能够直接推出什么?

A. 整个 FP32 输入域严格等价
B. 这些输入在所用比较规则下通过
C. 数学函数完全精确
D. 综合后所有时序都正确

答案：B：样本一致性、全域数学精度、有限精度语义和时序是不同验收项。

5.11 来源与阅读边界

- [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）
- [src-xls-float] XLS Floating-point routines

6 XLS 与流水线：从功能图到带时序的实现

模块 3 ·必须掌握·建议 55 分钟

XLS 是确定性的硬件编译工具链；AI 可协助写高层描述、准备测试和解释报告。自动生成的结果仍然依赖输入语义、时序约束、工具版本与下游实现。

先修： SFU 与 LUT：分清数学误差和硬件一致性

横向关联： 验证闭环：怎样证明 AI 写的实现符合规格 ·架构模型：先界定问题，再选择抽象层级

6.1 能力目标

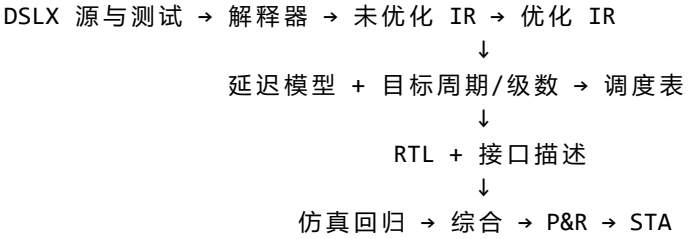
- 解释 DSLX、IR、调度和 RTL 的不同职责
- 区分时延、启动间隔和稳态吞吐
- 追踪背压下的事务、队列和寄存器占用
- 解释内部延迟模型为什么不能代替布线后 STA

6.2 为什么需要与系统位置

高层功能描述把算术意图与具体流水寄存器分开，便于探索结构。XLS从 DSLX 经 IR（Intermediate Representation，中间表示）、优化、调度到 RTL。纯函数适合无状态计算；proc 用于带状态和通信的描述。官方概览将 XLS 明确标为实验性项目，DSLX 兼容性需按版本核对。

视频中可见版本为 v0.0.0-10782-g5ddc35c4e。本课程引用的滚动文档访问于 2026-11-08；没有将当前 API 自动视为该 commit 的行为。复现时首先固定编译器 commit 与依赖。

6.3 数据与控制路径



每条边都可保留文件哈希与验证记录。IR 优化改变图结构，调度把操作分配到阶段，RTL 生成实现寄存器和接口。不要把 IR 当作已经完成物理放置的电路。

6.4 算法：依赖约束与资源占用

若运算 B 依赖 A，则 B 只能在 A 结果可用后执行；每级组合路径还需满足调度器的内部延迟模型。跨级存活的值需要寄存器。更多级数可能缩短每级路径，却增加寄存器、时钟树、控制和首结果等待。XLS 调度文档描述了周期、级数与寄存器成本的竞争。

配套教学模型采用另一种更小的 “弹性流水线”：每级存一笔事务、末级 ready 决定是否出队，空位向上游传播。这个模型教握手与占用，不声称复现 XLS 的所有接口或调度算法。

6.5 定量例题：L 与 II 不同

设理想流水线延迟 L=4 cycle、启动间隔 II=1 cycle、时钟 f=200 MHz。边沿 0 接受首笔，边沿 4 输出，所以首笔延迟 L/f=20 ns；稳态吞吐 f/II=200 M 笔/s。12 笔在边沿 0…11 进入，最后一笔在边沿 15 完成。从首笔接受到最后完成的物理总历时（makespan）为 (12−1)×II+L=15 cycle，即 75 ns；按该边界计算批次平均完成率为 12/15=0.8 笔/cycle。

若另选半开观察窗口 [0,16)，窗口时长是 16 cycle，窗口内完成 12 笔，得到 12/16=0.75 笔/cycle。这个观察窗口包含最后完成后的 1 个周期；16 个边沿记录本身不能推出物理历时 16 周期。两种有限窗口指标都不等于长期稳态 1/II=1 笔/cycle，报告必须写明起止边界。

增加到 8 级而 II 仍为 1，首结果变为 40 ns，稳态仍为 200 M 笔/s。提高时钟能否抵消延迟，要重新检查实际路径、寄存器开销和约束，不能单凭级数判断。

6.6 约束与反例

背压会拉长事务延迟并降低窗口吞吐；输入到达率大于服务率时，入口队列会增长。有限缓冲必须定义阻塞或丢弃策略。一个生产者在 valid=1、ready=0 时擅自改变 data，会破坏事务保持，而不是 “仅仅慢一点”。

调度时钟是目标或内部模型约束；最终 STA（Static Timing Analysis，静态时序分析）还取决于标准单元、连线、布局、时钟不确定性与角点。内部模型可行不保证布线后时序通过。

6.7 练习与工程交付

先在实验中预测 D=4、无停顿时首笔边沿 4 输出、12 笔最后边沿 15 完成；分别写出物理历时 15 cycle 与观察窗口 [0,16) 内的完成率。把下游每 5 拍阻塞 2 拍，逐拍找出事务和寄存器等待的位置。最后设置下游永不 ready，解释 “未完成” 为什么不能报告为 0 延迟。工程交付应包含源、IR、schedule、RTL、工具版本、约束与报告，而不是只给一个 Verilog 文件。

6.8 能力训练与完整答案

练习 1 •补全计算

题目：L=6、II=2、f=250 MHz，首结果延迟与稳态吞吐是多少?

补全步骤与提示：时间 =L/f；吞吐 =f/II。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 24 ns；125 M 笔/s。忽略停顿、输入不足与填充排空。

自评量表

- 延迟公式
- 吞吐公式
- 单位
- 统计边界

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 •画时序题

题目：画出 3 级流水线，下游在边沿 3、4 不 ready 时，前 4 笔事务的状态。

补全步骤与提示：逐拍从末级向前判断谁能移动；再决定入口接收。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 使用 pipeline 实验设 D=3、P=5、S=2 时需注意阻塞发生在余数 0、1；若指定边沿 3、4 则手动平移阻塞相位。核心要求是旧事务不被覆盖、输出顺序不变、空位才能接收。

自评量表

- 显式阻塞相位
- 保持数据
- 无丢失重排
- 解释入口队列

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 •迁移题

题目：若乘法器每 3 拍才能接受一个输入，如何扩展目前每级 1 拍的模型？

补全步骤与提示：区分占用时间和结果延迟。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 为乘法资源建 next_available 或服务令牌，单独定义 II=3 与结果 ready 时刻；上游等待、输出队列与共享使用者都按事件更新。不可只给总延迟加常数。

自评量表

- 区分 II 与 L
- 资源占用状态
- 依赖与队列
- 验证边界用例

评分：满分 4 分；开放答案采用自评，不自动判定正确。

6.9 快速检查

1. 流水线从 4 级增加到 8 级且时钟、II 不变，哪个结论成立?

A. 稳态吞吐必然翻倍
B. 首结果延迟增加，稳态吞吐未必改变
C. 功耗必然下降
D. 面积必然下降

答案：B：L、II 与时钟频率是三个不同量。更多流水级不能单独推出 PPA 收益。

6.10 来源与阅读边界

- [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）
- [src-xls-overview] XLS: Accelerated HW Synthesis - Overview
- [src-xls-quickstart] XLS Tools Quick Start
- [src-xls-dslx] XLS DSLX Reference
- [src-xls-scheduling] XLS Pipeline Scheduling
- [src-xls-codegen] XLS Codegen Options

7 验证闭环：怎样证明 AI 写的实现符合规格

模块 4 ·必须掌握·建议 65 分钟

AI 能快速提出实现，验收必须依靠规格和独立证据。贯穿本章的原创例子是 8 位无符号饱和加法：输入之和超过 255 时输出 255，而不是回绕。

先修：先把问题写成可验收的规格

横向关联：SFU 与 LUT：分清数学误差和硬件一致性 ·XLS 与流水线：从功能图到带时序的实现 ·工程 Agent：状态、上下文与可验证交付

7.1 能力目标

- 区分编译通过、功能正确、时序通过与系统可用
- 画出刺激、DUT、独立参考和记分板的数据路径
- 设计能击穿错误实现的边界测试
- 用机器可读证据约束 AI 修复并设置退出条件

7.2 为什么需要：生成器与检查器不能共同猜答案

把 DUT（Device Under Test，被测设计）想成 CPU 数据通路，记分板想成独立执行参考 ISA 的对照机。编译器只能排除某些语法和类型问题；它不知道你要的是饱和还是回绕。两份代码来自同一段错误解释，即使逐位相同，也只能说明共同犯错。

录屏中对 Python、DSLX 和 RTL 的阶段性对照，是值得保留的工程骨架。但数学逼近是否够精确、两种实现是否一致，是两个不同问题；把同一个错算法搬到每一层，仍可能全部一致。

7.3 系统位置与数据路径

```
冻结规格 → 测试输入 → DUT → 实际输出 ┐
       └→ 独立整数参考 → 期望输出 ──┤ compare → PASS/FAIL
工具版本、随机种子、输入哈希、日志 ──┘
FAIL → 分类与最小反例 → AI 提交局部修复 → 全部回归
```

- 冻结验收条件：输入 a,b 为 0…255，输出 min(255,a+b)，复位清除 valid，背压时保持数据。
- 只读参考模型和回归用例；AI改它们需要单独解释规格变更，不能为了过测试降低预期。
- 错误类别先分：环境/编译、数值、握手、复位、未知态、超时。每类对应不同修复动作。

7.4 状态变化：一个弹性输出寄存器

接口使用 valid/ready（有效/就绪）握手：只有同一时钟边沿两者都为 1，事务才发生。单级模块令 in_ready = !out_valid || out_ready。若输出寄存器空，或者旧输出即将被消费，就能接收新输入。否则保持 out_valid 与 data，禁止新输入覆盖旧结果。

在同步电路中，“某条信号出现上升沿”与“所有下游非阻塞赋值完成”不是同一观察时刻。cocotb 2.0.0 的时序文档区分边沿事件和 ReadOnly 稳定采样阶段；实际测试还需核对仿真器语义。

来源：cocotb Timing Model，Values Change / End of Time Step。本仓库实测采用 SystemVerilog testbench，避免把未运行的 cocotb 示例说成已验证。

7.5 约束与验收阶梯

- lint：位宽、隐式网线、风格与可疑结构；不是完整功能证明。
- 仿真：只对运行的输入和序列提供证据。固定随机种子是复现条件，不是覆盖充分性的证明。
- 形式化：在明确假设、状态范围和属性下证明；过强假设可能排除真正缺陷。
- 综合 / STA：检查映射后的实现与时序约束；功能仿真 PASS 不等于时序收敛。
- 系统性能：微模块的吞吐与整机延迟不相等，还要纳入调度、内存和资源冲突。

7.6 完整例题：4 个普通输入为什么漏掉溢出

规格 y=min(255,a+b)。错误实现 y=(a+b) mod 256。对 (1,2)、(10,20)、(40,60)、(0,0) 两者都相等，4/4 PASS。加入 (255,1) 后，期望 255，实际 0；加入 (128,128) 后，期望 255，实际 0。反例直接指向截断前没有检测第 9 位进位。

全部组合输入共有 $2^8 \times 2^8 = 65{,}536$ 对；单凭 5 个边界向量只覆盖 5/65,536≈0.00763% 的枚举点。这个比例不是功能覆盖率，更不是“发现错误的概率”。时序输入还包括任意长握手和复位序列，不能由组合全枚举覆盖。

```
sum9 = zero_extend(a, 9) + zero_extend(b, 9)
y = 255 if sum9 > 255 else sum9
assert hold_output_when(valid && !ready)
assert compare_only_on(valid && ready)
```

7.7 取舍：快反馈与独立性

快速 lint/单元用例放前面，昂贵回归和物理实现放后面，可以减少无效等待。局部修复也必须跑相关全回归，否则可能修复溢出却破坏停顿。固定预算，例如最多 3 次修复、每次 10 分钟、失败保留最小反例，是教学配置而非最佳实践数字。

AI 可解释日志、提出反例和补充测试，但最终接受条件由确定性工具与审查约束。OpenAI 的修复闭环示例也强调可执行反馈；将其迁移为 RTL 闭环是本课程的工程设计。官方示例。

7.8 失败与误解：把错误复制成 golden

如果参考模型也使用 (a+b)&255，那么 (255,1) 的“期望”也变成 0，测试仍 PASS。实验室允许切换这个错误参考，观察 PASS 与真实符合规格的差异。修复办法不是再问一个模型投票，而是用明确规格推导参考式、保留反例、审查两个实现的共同来源。

不要把局部仿真的通过率命名为流片成功率。也不要让 agent 修改测试数据直到变绿，而不留下规格变更记录。

7.9 练习路线与可复现实验

先完成“独立 oracle”纸面推演：预测普通输入能否识别错误，再分别列出边界输入，以及参考模型复制实现时的结果。随后运行源码 examples/sat-add：既检查数值输入域，也检查背压与复位。运行证据记录在 运行验证报告；没有列在报告中的工具链未声称执行。

7.10 能力训练与完整答案

练习 1 •补全步骤

题目：补全饱和加法参考模型，并给出两个溢出反例。

补全步骤与提示：先扩展至 __ 位相加，再与 __ 比较；超出时输出 __。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 9 位；255；255。反例可用 (255,1)、(128,128)。

自评量表

- 明确无符号输入
- 先扩展再求和
- 输出饱和而非回绕
- 给出两个有预期值的反例

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 ·调试题

题目：out_valid=1、out_ready=0 时，AI 生成的模块仍更新 data。画两拍状态并解释错误。

补全步骤与提示：先写消费者是否接受；再判断生产者能否覆盖旧值。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 没有发生输出握手，旧事务仍待消费；必须保持 valid 和 data，并拉低 in_ready。下一拍恢复 ready 后才能消费旧输出与接收新输入。

自评量表

- 识别未握手
- 画出旧事务保持
- 指出覆盖导致丢数据
- 提出 in_ready 控制

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 ·独立设计

题目：设计一个 16 位带符号饱和乘法的验证计划。不能复用本例的无符号阈值。

补全步骤与提示：先决定数学乘积宽度、输出范围与舍入，再列输入分区。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 完整乘积用 32 位有符号数，输出限幅到 [-32768,32767]。分正负、零、极值与阈值附近，检查符号扩展；另加复位/背压序列，独立参考禁止使用 DUT 的截断表达式。

自评量表

- 定义范围与符号
- 独立参考和中间位宽
- 边界与反例
- 时序和回归证据

评分：满分 4 分；开放答案采用自评，不自动判定正确。

7.11 口述与追问

为什么两个阶段输出一致仍不能证明数学精度达标?

参考回答：一致性证明相互实现的一致，不证明它们与数学目标足够接近。要分别验证目标误差和实现等价；独立参考、特殊值和范围条件都要固定。

- 若两个参考都调用同一错误库怎么办?
- 怎样把最坏输入固化为回归?

7.12 快速检查

1. 4 个普通输入通过后，下一个最有价值的测试是?

A. 再测 1+1
B. 检查 255+1 与停顿时数据保持

C. 只提高随机种子数量
D. 让同一个模型确认代码

答案：B：溢出与握手边界分别针对数值和时序故障。随机测试有价值，但不能替代有机制依据的定向反例。

7.13 来源与阅读边界

- [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）
- [src-cocotb-2] cocotb 2.0.0 Documentation
- [src-cocotb-timing] The Timing Model - cocotb 2.0.0
- [src-verilator-overview] Verilator Overview
- [src-openai-repair] OpenAI - Build iterative repair loops with Codex

8 架构模型：先界定问题，再选择抽象层级

模块 5 ·必须掌握·建议 70 分钟

录屏的大部分时间用于讨论架构建模：模型需要多细、哪些参数可信、如何从算子映射到硬件资源。一个快速模型可以指导架构取舍，但必须清楚它没有模拟什么。

先修：先把问题写成可验收的规格

横向关联：XLS 与流水线：从功能图到带时序的实现 ·算子融合与 L1：数据搬运为什么支配结果 ·模型验证与校准：把“看起来合理”变成可检查

8.1 能力目标

- 比较解析、事件、周期与功能模型的职责
- 用 Roofline 手算计算/传输下界
- 设计从算子图到资源事件的表示
- 识别性能模型中峰值、并行与等待的混用

8.2 为什么需要：不同问题要求不同模型

在 RTL 尚未定稿时，仍要回答“加 L1 还是加算力”“融合是否划算”“SFU 是不是瓶颈”。逐条指令、逐周期模拟可能太慢；只做总量除峰值又看不见依赖和竞争。模型应从需要回答的设计问题出发。

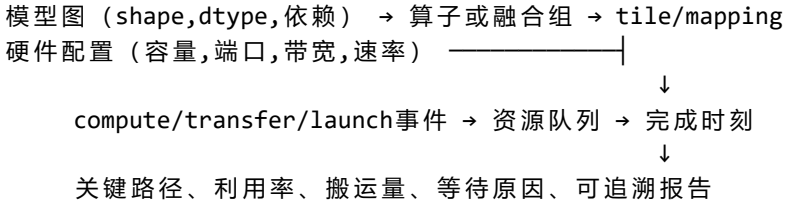
类似 CPU 的 ISA 模型与微架构模拟器：前者检查可见计算，后者解释资源与时间。但真实架构模型可能介于两者之间，不能仅凭“模拟器”名称推断它有周期精度。

8.3 系统位置与抽象层级

- 功能参考：给定输入算对输出，通常不预测周期。
- 解析模型：用运算量、访问量、带宽、启动成本推导上下界，适合快速筛选。
- 离散事件模型：记录就绪、开始、完成、资源释放，适合依赖和排队。
- 周期模型：每拍更新微结构状态，能回答更细的仲裁/冲突问题，前提是这些细节被实现。
- RTL/物理实现：检查接口与可综合实现，仍需独立功能和物理验收。

录屏中“无需计算所有精确数值，先验证瓶颈”的建议，是针对架构探索的取舍，不等于数学证明，也不能把一句“可覆盖大部分验证”当作经过测量的覆盖率。

8.4 对象与数据/控制路径



至少为每个事件记录 id、依赖、资源、工作量、单位、来源和开始/结束时刻。资源包含计算单元、本地读写端口、DMA、互连与外部通道。tensor 名字相同但 shape 不同，不能共享未经核对的成本。

8.5 定量模型与单位

令 O 为总运算量 (op)，P_cap 为同一统计范围内不能超过的计算速率 (op/s)，Q 为某存储层级的访问字节 (byte)，B_cap 为该层级不能超过的带宽 (byte/s)。则 T_c,min=O/P_cap、T_m,min=Q/B_cap，运行时间必不小于 max(T_c,min,T_m,min)。这只是必要下界；依赖、启动与竞争可能使实际时间更长。运算强度 I=O/Q(op/byte)，相应性能上界为 min(P_cap,I×B_cap)。Roofline 原始论文提供这一界限与瓶颈分析思路。

若用 B_model=η×B_peak 表示假设的持续有效带宽，T_m,est=Q/B_model 就是该效率假设下的传输估算，不能自动成为真实硬件的严格下界。一次微基准测得的平均带宽也不是所有负载的硬上限。采用测得的可持续带宽构建经验屋顶线时，应说明测试负载与适用范围。

在本章简单模型中，固定服务速率后，完全重叠估计 T_est=max(T_c,est,T_m,est) +T_launch；完全串行估计 T_est=T_c,est+T_m,est+T_launch。这两个等式依赖各自的排程假设。只有同一统计范围的 O、Q、P、B 才能放进一个式子：整层 O 不能配单 tile 的 Q；MAC 计几 op、GB 还是 GiB、每核还是全芯片，都应写在报告上。

8.6 完整例题：翻倍算力为什么没加速

原创教学负载：O=512 Gop、Q=64 GB，计算模型假设以 128 Gop/s 持续服务；外部带宽峰值 B_peak=32 GB/s，假设效率 η=0.5，故 B_model=16 GB/s。计算估计 T_c=4 s，传输估计 T_m=4 s。完全重叠并加 5 ms 启动得到 4.005 s；完全串行得到 8.005 s。这是所选效率与排程假设下的模型结果，不是硬件严格下界或实测。若仅用不能超过的 32 GB/s 峰值，能推出的传输下界是 2 s。

把计算模型速率提高到 256 Gop/s，只把 T_c 降到 2 s，重叠模型总时长仍为 4.005 s。再把 Q 减半到 32 GB，T_m 也变为 2 s，结合后总时长为 2.005 s。单项优化可能被另一资源遮住；应分别改变参数观察瓶颈迁移，并记住持续计算速率本身也是模型假设。

8.7 从静态式子到事件状态

对非流水、非抢占的独占资源，事件开始时间 =max(所有依赖的数据就绪时刻，资源可用时刻)，结束时间 = 开始时间 + 服务时间；结束后释放该资源。多资源事件还必须同时获得所需资源。多个独立事件可以重叠，但共享端口需要排队或明确的带宽分享规则。

对流水资源，结果延迟 L 与再次发射间隔 II 必须分开：next_issue=start+II，result_ready=start+L。依赖者等 result_ready，独立的新事务则可在 next_issue 发射，还要满足内部容量、端口和背压条件。例如 L=4、II=1 的理想单元可每拍接收一笔；若错误地直到结果完成才释放发射机会，就会把 II 错建成 4。不能用一个 “busy 至完成” 的状态表示所有流水资源。

资源模型可分固定服务与随 shape 变化的成本。TopK 等操作还可能依赖算法和数据分布；服务、排队等待、发射限制及结果就绪应分别记录，不能用一个与 shape 无关的 “每元素周期” 解释所有情况。

8.8 取舍与反例

解析模型的快来自省略细节。它适合上下界和灵敏度，不适合凭空预测 bank 地址冲突。周期模型若配置过时、端口共享关系错误，也可能更慢但不更准。增加细节前应说明它能改变哪个设计决策，并准备校准数据。

反例：4 个并行请求访问同一单端口 bank。总容量足够、总字节很少，Roofline 仍可能乐观；地址映射与仲裁决定串行服务。另一个反例是启动开销主导的小算子，总 op 很少也不代表时间可忽略。

8.9 练习与迁移

完成 Roofline 实验后，画一个 “DMA 装入 → 计算 → 写回” 的事件图；再让两个 tile 共用 DMA，比较完全串行、双缓冲和独立 DMA 三种条件。最后提出一个能区分模型偏差来自带宽还是启动开销的微基准。

8.10 能力训练与完整答案

练习 1 ·补全计算

题目：保持默认模型，仅把峰值计算速率从 128 提升到 256 Gop/s，理想重叠时间如何变？

补全步骤与提示：分别求 Tc、Tm，不能将原总时间直接除 2。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 Tc=2s，Tm=4s，总计 4.005s。带宽成为唯一主瓶颈。

自评量表

- Tc 正确
- Tm 不变
- max 而非 sum
- 启动开销

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 ·调试题

题目：模型把每核带宽乘以核数后性能特别好，实际没有提升。给出三个排查点。

补全步骤与提示：画出共享资源；核对单位；检查通信/同步。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 检查带宽究竟是共享总量还是每核；是否把峰值当有效；跨核/内存争用是否建模；确认负载能并行而非串行关键路径。

自评量表

- 共享关系
- 有效/峰值
- 依赖与同步
- 具体测量方法

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 ·陌生场景

题目：为一个多请求共享压缩引擎建立最小事件模型，并声明不能回答的问题。

补全步骤与提示：列到达、队列、启动、服务、完成事件。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 用输入大小和压缩模式定义服务成本；维护单引擎可用时间与队列；独立测量启动/吞吐参数。没有数据分布模型时不能可靠预测压缩率和数据相关服务差异。

自评量表

- 状态与事件
- 依赖与资源
- 参数来源
- 明确边界

评分：满分 4 分；开放答案采用自评，不自动判定正确。

8.11 快速检查

1. 分析模型没有 bank 地址映射，能够给出准确冲突周期吗？

A. 可以，容量足够即可
B. 不能，应补充映射/仲裁，或明确只给理想性能上界与时间下界
C. 可以由 LLM 补一个默认值
D. 把频率乘 4 即可

答案：B：模型没有的状态不能从总量公式中自动恢复。需要增加可验证机制，而不是增加未经校准的小数位。

8.12 来源与阅读边界

- [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）
- [src-roofline] Roofline: An Insightful Visual Performance Model for Multicore Architectures

9 算子融合与 L1：数据搬运为什么支配结果

模块 6 ·必须掌握·建议 65 分钟

录屏在模型图、指令拆分和 L1 配置之间反复往返。这里的主线是：先保留算子语义，再决定数据怎么切、放在哪里、何时复用；融合收益来自减少搬运，但可能被容量或端口代价抵消。

先修：架构模型：先界定问题，再选择抽象层级

横向关联：SFU 与 LUT：分清数学误差和硬件一致性 ·模型验证与校准：把 “看起来合理” 变成可检查

9.1 能力目标

- 从张量 shape 与 dtype 计算存储量
- 计算中间结果写回/重读的流量
- 分析融合的容量、活跃区间与端口代价
- 比较独立算子与融合实现时保持同一计量边界

9.2 为什么需要与系统位置

一个计算图有很多看起来简单的算子。逐个运行时，中间张量可能写入外部内存，再由下一算子读回。融合把这些中间量留在寄存器或本地 buffer，减少往返。和 CPU 编译器融合循环的直觉相似，但加速器还受 tile 尺寸、专用引擎、片上存储和 DMA 约束。

9.3 对象与数据路径

```
未融合：读X → 算子A → 写Y → 读Y → 算子B → 写Z
融合：  读X → [A → 本地Y → B] → 写Z
映射约束：tile shape、dtype、生命周期、buffer、读写端口
```

图的节点表示语义，不一定是一条 ISA 指令；mapping 描述如何用硬件资源实现。把一个复杂算子拆成基础运算可以揭露 SFU 或归约成本，但过早拆细也会让分析冗长、重复计算搬运。应保留语义层与实现层之间的映射。

MLIR Transform 官方教程的结构化操作与 Loop Fusion 部分提供了保留索引/迭代语义并表达融合的具体例子。这里的容量、端口和外部流量算例是课程增加的工程分析；IR 合法转换本身不证明目标硬件加速。

9.4 算法：容量靠活跃区间计算

工作集不是把所有张量容量简单相加，也不是只看最大一个张量。应在执行时间线上计算同时活跃的输入、输出、累加器和临时 buffer。双缓冲可能把某些输入/输出 buffer 占用加倍；对齐、padding 和元数据也需要空间。

如果 buffer 复用是推测而未实现，就不能在模型中提前 “免费释放”。每次写入、读出和释放都应关联事件或明确的静态生命周期。容量通过后还要核对同拍端口需求与 bank 映射。

9.5 完整例题：融合省了多少搬运

原创教学张量 Y 有 1,048,576 个 BF16 元素，每个 2 byte，所以容量为 2,097,152 byte=2 MiB。未融合的一次写 Y 和一次读 Y 共 4 MiB。假设持续有效外部带宽为 16 GB/s，忽略启动与其他开销，传输时间估计为 $4\times2^{20}/(16\times10^{9})=0.262144$ ms；只有把 16 GB/s 明确作为不可超过的带宽上限时，这个值才是相应下界。融合可消去这部分外部流量，却不保证端到端恰好快 0.262144 ms，因为传输可能原已与计算重叠。

设本地容量 4 MiB，X、Y、Z 各 2 MiB，朴素同时保留共 6 MiB，不可行。若按半张量 tile 切分，并证明 Y 可消费后立即释放，就可能降低峰值活跃量；若还要双缓冲输入，必须重新计算，不能默认依旧放得下。

9.6 约束：精度、归约与数据相关操作

重排浮点归约顺序可能改变舍入；softmax 等归约有跨元素依赖，不能像逐元素激活一样任意独立切块。融合的语义验证要检查边界、shape、mask 和特殊值。TopK 或稀疏路由还可能改变访问局部性和负载均衡。

把所有操作都折算成 MAC 吞吐，会漏掉 Exp、除法、归约、scatter/gather 等不同资源。相同 op 计数并不意味着同一服务速率。

9.7 取舍与失败案例

融合收益：减少中间写回、减少启动次数、改善复用。成本：工作集增加、实现复杂、并行机会减少、寄存器或 L1 压力、调试和验证边界扩大。强行融合可能引发 spill 到外存，反而比未融合更慢。

反例：两个并行分支共享 Y。把 Y 仅保留在某一融合组内部，却忘记另一个消费者，会漏掉必要搬运或重复计算。正确方法是对图的所有消费者和生命周期做检查。

9.8 练习路线

先跟着 2 MiB 中间张量算例算流量，补全 FP32 版本的字节数，再为 “矩阵乘 → 偏置 → 激活 → 归约” 列两种融合候选。对每种方案分别给语义、容量、端口、启动和搬运证据；不能只给一张更短的算子列表。

9.9 能力训练与完整答案

练习 1 •补全计算

题目：上述 1,048,576 元素改用 FP32，Y 容量与写读流量分别是多少?

补全步骤与提示：每元素 4byte；流量含一次写和一次读。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 Y=4 MiB，往返 8 MiB；在持续有效 16 GB/s 假设下，忽略额外开销的传输估计为 0.524288 ms。不能直接视为端到端节省量，也不能把假设速率自动视为真实硬上限。

自评量表

- dtype 字节
- 容量
- 往返次数
- 单位和边界

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 •画图题

题目：画 X/Y/Z 的活跃区间，解释 4MiB 容量何时能容纳三个各 2MiB 张量。

补全步骤与提示：允许分块或复用，但必须标注何时可释放。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 如果三个完整张量同时活跃则 6MiB，不可行。可分块到各 1MiB，或在消费者完成后复用 buffer；任何共享消费者和双缓冲都需重新计算峰值。

自评量表

- 生命周期
- 峰值而非总和
- 复用条件
- 额外消费者/双缓冲

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 ·设计题

题目：为带 mask 的 softmax 提出融合方案和一个不等价反例。

补全步骤与提示：考虑 max/sum 归约与全被 mask 的行。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 不能每 tile 独立归一化后直接拼接；需跨 tile 维护一致的 max 与归一化量，规定全 mask 语义并测试极大/极小输入。

自评量表

- 跨 tile 依赖
- 数值稳定
- mask 边界
- 独立验证

评分：满分 4 分；开放答案采用自评，不自动判定正确。

9.10 快速检查

1. 融合消除了 4 MiB 流量，能直接把端到端时长减去 Q/B 吗?

A. 总能
B. 不能，需要看重叠、关键路径和新开销
C. 只要 AI 说可以就行
D. 只取决于算子数量

答案：B：Q/B 在本题给定的持续带宽假设下是传输时间估计；只有 B 为速率上限时才构成相应下界。被计算隐藏的传输消除后，关键路径可能不变。

9.11 来源与阅读边界

- [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）
- [src-roofline] Roofline: An Insightful Visual Performance Model for Multicore Architectures
- [src-mlir-transform] MLIR Transform Tutorial - Chapter 0

10 模型验证与校准：把“看起来合理”变成可检查

模块 7 ·深入选学·建议 65 分钟

模型能运行、表格能出数，不代表能支撑架构决策。先检查代码是否实现所选模型，再检查该模型在目标负载范围内是否足够贴近参考。

先修：架构模型：先界定问题，再选择抽象层级 ·验证闭环：怎样证明 AI 写的实现符合规格

横向关联：算子融合与 L1：数据搬运为什么支配结果 ·工程 Agent：状态、上下文与可验证交付

10.1 能力目标

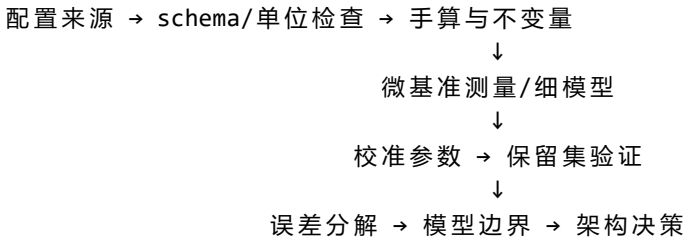
- 区分模型 verification 与 validation
- 用守恒与极限用例发现建模错误
- 设计区分启动开销和服务速率的微基准
- 保留校准集之外的验证任务并报告误差分布

10.2 为什么需要与系统位置

Verification（实现检查）问“模型代码是否正确实现定义”；validation（有效性验证）问“这个定义是否足够反映目标系统”。给吞吐公式写单元测试只能解决前者的一部分；与微基准或更细模型比较解决后者。二者都不能靠报告排版精美来替代。

Timeloop/Accelergy 官方概览列出使用周期仿真、设计数据与综合结果核对模型的途径；Accel-Sim 作者框架提供 trace、性能模型和硬件关联比较组件。本章的独立手算、保留集与误差分层是据此展开的教学验证方案，不宣称复现了这些项目的误差数字。

10.3 对象与控制路径



对每个参数记录值、单位、层级、来源、适用 shape/dtype 和可信度。实测有效带宽与理论峰值应是两个字段，不能用同名变量覆盖。发生参数更新时，报告必须绑定配置哈希。

10.4 算法：先用不变量，再用拟合

- 事务守恒：输入接受数 = 输出完成数 + 在途数 + 明确取消数。
- 资源互斥：独占资源上任意两个事件区间不能重叠。
- 依赖正确：消费者开始不早于生产者数据就绪。
- 容量约束：每时刻活跃 buffer 不超过配置容量。
- 维度一致：byte/(byte/s)=s，cycle/(cycle/s)=s。
- 单调性仅在固定机制下使用：单纯提高一个上界不应使该解析子模型变慢；真实系统改变策略时不能机械套用。

10.5 完整例题：用两次测量拆出启动开销

教学微基准同一串行引擎：处理 1 MiB 耗时 0.20 ms，处理 5 MiB 耗时 0.60 ms。模型 $T=T_0+Q/B$。相减得到 4 MiB/0.40 ms≈10.48576 GB/s；代回得到 T_0=0.10 ms。若直接用 1 MiB/0.20 ms 得到 5.24288 GB/s，会把启动开销错误地归入带宽。

再用独立的 9 MiB 保留样本，模型预测 1.00 ms。若实测 1.40 ms，应检查缓存层级、竞争、分块边界或非线性，而不是只把 B 继续调到让所有点看似接近。以上数字为教学构造，未声称测量本机硬件。

10.6 约束与误差报告

报告每个样本的预测与参考、绝对偏差、相对偏差和负载类别。平均误差可能掩盖长尾；小参考值会放大相对误差，应分组报告或声明阈值。校准集用于选参数，保留集用于评估泛化，不能把同一批样本两种用途混用。

对没有数据的参数可做区间敏感度：例如 η 在 0.4…0.8，输出延迟区间如何变化。区间是情景估算，不是统计置信区间；不能在没有随机模型时随意标 “95% 置信度”。

10.7 取舍与反例

反例一：模型漏记写回，拟合时把带宽调低补偿。对写回比例不同的新负载立刻失败。反例二：把顺序依赖的多个算子时长取 max，模型异常乐观；调低算力也可能暂时拟合，但机制依然错误。

更多精度并非永远必要：若两种架构收益区间高度重叠，当前模型不足以做决定，应追加能区分候选的测量。若差异远大于模型误差，快速分析可能已经足够用于早期筛选。

10.8 练习与交付证据

先用上述两点求 T_0 与 B，再写一个自动单元测试保护单位换算。设计第三点验证，把结果记录为模型版本、配置、输入、参考、误差、判断和后续动作。对未知 bank 冲突，不填虚构数字；列为下一层模型待验证项。

10.9 能力训练与完整答案

练习 1 •补全计算

题目：若保留集 9MiB 实际为 1.40ms，而预测 1.00ms，相对参考值的偏差是多少？

补全步骤与提示：分母用参考 1.40ms。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 $(1.00-1.40)/1.40\approx-28.57\%$，表示低估。若以预测作分母会得到 40%，必须声明口径。

自评量表

- 正确分母
- 符号
- 数值
- 统计边界

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 •反例题

题目：模型漏掉输出写回但调低带宽后拟合通过。如何揭露？

补全步骤与提示：构造读写比例不同、总输入相同的任务。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 保持输入读量，改变输出大小；若真实时长随写回增长但模型不变，则机制缺失。独立报告读写字节并检查资源事件。

自评量表

- 可区分实验
- 控制变量
- 计量读写
- 不以再拟合掩盖

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 ·设计题

题目：为数据相关 TopK 模型制定校准与保留集方案。

补全步骤与提示：同时改变 shape、k、分布和竞争。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 按 shape/k/数据分布分层；分别测启动、计算与访存；保留未参与拟合的组合；报告尾部误差与不支持区域，不能只用平均每元素周期。

自评量表

- 分层变量
- 独立保留集
- 资源分解
- 边界与误差

评分：满分 4 分；开放答案采用自评，不自动判定正确。

10.10 快速检查

1. 同一批样本拟合后误差为零，说明什么？

A. 模型对所有负载准确
B. 只说明拟合能解释这批数据，仍需保留集与机制检查
C. 不再需要单位检查
D. 比真实测量更可信

答案：B：拟合可能补偿遗漏机制，零训练误差不是泛化和因果解释的保证。

10.11 来源与阅读边界

- [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）
- [src-roofline] Roofline: An Insightful Visual Performance Model for Multicore Architectures
- [src-timeloop-overview] Timeloop/Accelergy Overview
- [src-accelsim] Accel-Sim author framework repository

11 PPA 与候选筛选：证据、约束和 Pareto

模块 8 ·必须掌握·建议 55 分钟

PPA（Power, Performance, Area，功耗、性能与面积）不是一个越小越好的单分数。先排除不正确或不满足时序/精度的候选，再在相同边界下比较成本和收益。

先修： XLS 与流水线：从功能图到带时序的实现 ·验证闭环：怎样证明 AI 写的实现符合规格

横向关联： 模型验证与校准：把 “看起来合理” 变成可检查 ·选择 AI 芯片工程 Flow：按任务、证据与成本比较

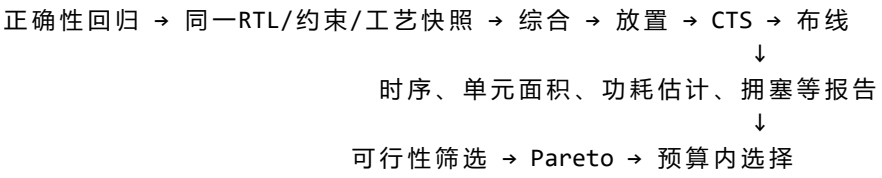
11.1 能力目标

- 核对同一 PPA 比较的工艺、角点、活动率和统计阶段
- 计算功耗和面积相对变化
- 先筛可行候选，再识别 Pareto 支配
- 区分研究 PDK、工具估计和产品签核

11.2 为什么需要与系统位置

一个错误电路可以非常省电，一个未收敛时序的电路也可能面积很小。优化不能绕过正确性门槛。AI 适合提出候选、解释报告、规划下一轮实验；工具和冻结的验收规则决定候选是否可接受。

11.3 对象与数据路径



OpenROAD Flow Scripts 官方文档描述从 RTL 到 GDSII 的自动实现流程。输入除了 RTL，还包括 SDC 约束、库与物理信息。生成 GDS 只是流程产物，不能替代特定代工厂和项目所需全部签核。ORFS 流程文档。

11.4 录屏案例：把结果和边界一起读

约 05:00-07:00 的幻灯片列出三轮估计总功耗 98.97、49.76、44.70 mW；原始与最终单元面积 10,620 与 10,865 μm^2；流水级依次 14、16、17。显示的 setup slack 依次为 −461.920、−100.206、+53.866 ps：前两轮不满足所显示的 setup 约束，最终该项转正。因此这不是两个同样满足 200 MHz 时序的可行实现之间的纯功耗比较；正 setup slack 本身也不证明 hold、所有路径/角点或全部签核通过。

幻灯片标注 ASAP7 RVT SS、200 MHz，功耗使用 OpenSTA 默认活动率，未使用 workload SAIF/VCD。这里只记录画面可见案例，未取得原始报告独立复现。末轮相对首轮估计功耗变化 =(44.70−98.97)/98.97≈−54.83%；面积变化 =(10865−10620)/10620≈+2.31%。侧栏自动纪要出现约 50.7% 的不同说法，不能替代原表和重算；这些结果也不能据此推广为实际应用功耗减半。

11.5 比较约束与统计边界

- 固定功能、输入范围、精度、接口和负载。
- 固定工艺库、角点、供电、频率/SDC、工具版本与流程阶段。
- 明确功耗是默认活动率估计、工作负载 VCD/SAIF 估计，还是硅后实测。
- 区分单元面积、核心面积、芯片面积；利用率变化可能改变布局空间。
- 记录流水级、II、启动开销与有效吞吐；局部时钟指标不是整机性能。
- 失败运行、未收敛约束、不同配置的报告不得混入同一均值。

11.6 算法与完整例题：Pareto 不是唯一冠军

若 A 在所有比较指标上不比 B 差，并至少一个更好，则 A 支配 B。先满足最大误差和最大延迟硬约束，再比较面积、路径延迟和功耗。配套教学模型中的数值全部是人工教学样例，与录屏数据分开。

教学基线面积 100、延迟 1.8ns、功耗 50mW；均衡候选 90、1.5ns、44mW 在三项均更优，因而支配基线。深流水候选 122、1.1ns、56mW 速度更快但成本更高，两者可能同在前沿。若时序门槛 1.2ns，均衡候选被排除；若门槛 0.8ns，则没有可行解，不应硬选“最佳”。

11.7 取舍、反例与验证

为了优化报告数字而改低频率或活动率，属于改变比较条件，不是同条件优化。多个随机种子或布局波动需要报告分布和最差值。未校准的面积/功耗代理可用于快速排序，但最终比较应回到相同工具和约束运行。

研究 PDK 是探索和方法验证的工具，不能默认等价于某商业工艺的可制造结果。视频提出“预算内最佳已验证候选”的方向是合理目标，但不是搜索空间全局最优证明。

11.8 能力训练与完整答案

练习 1 •补全计算

题目：用 44.70 与 98.97mW 计算相对下降比例，分母选谁?

补全步骤与提示：相对原始基线：1− 新/旧。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 1−44.70/98.97≈54.834798%，按两位小数为 54.83%。分母是 98.97。它比较的是所显示的估计功耗；原始候选 setup slack 为负，不是同样满足 200 MHz 时序的可行基线。

自评量表

- 正确基线
- 算式
- 结果
- 估计功耗边界

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 •反例题

题目：候选 A 比 B 功耗低 20%，但 A 是 100MHz、B 是 200MHz。如何重做比较?

补全步骤与提示：先固定目标；可比较同频，或同吞吐下能量。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 统一频率/电压/活动/约束重跑；若比较每任务能量，需记录完整任务时长与功耗积分，不能只比 mW。

自评量表

- 发现频率混用
- 统一条件
- 能量与功率区别
- 重跑证据

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 ·独立设计

题目：给 5 轮 AI 候选制定停止规则与交付目录。

补全步骤与提示：考虑预算耗尽、无可行解、重复候选和目标达成。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 固定 5 轮/资源预算；失败归档，重复哈希不重跑；达到阈值保留前沿候选；无可行解明确报告。交付 spec、源码/配置/工具哈希、测试、PPA 报告、选择理由。

自评量表

- 有限预算
- 可行性门槛
- 去重与失败
- 可审计交付

评分：满分 4 分；开放答案采用自评，不自动判定正确。

11.9 快速检查

1. 两候选功耗分别 44 与 50mW，哪项信息缺失会使比较失去意义?

A. 按钮颜色
B. 活动率/负载与频率是否一致
C. 源码行数
D. 模型回答长度

答案：B：功耗依赖活动率、频率、电压、库与统计范围。缺少统一条件，差值可能完全来自测量口径。

11.10 来源与阅读边界

- [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）
- [src-orfs-flow] OpenROAD Flow Scripts - RTL-to-GDSII Flow
- [src-orfs-tutorial] OpenROAD Flow Scripts Tutorial

12 工程 Agent：状态、上下文与可验证交付

模块 9 ·必须掌握·建议 55 分钟

让 AI 帮你编写 RTL、算子模型或仿真器，关键不在于对话轮数，而在于每次改变都留下可复查的输入、工具结果与判断依据。本章把 agent 当作工程控制系统来设计。

先修：验证闭环：怎样证明 AI 写的实现符合规格

横向关联：架构模型：先界定问题，再选择抽象层级 ·算子融合与 L1：数据搬运为什么支配结果 ·PPA 与候选筛选：证据、约束和 Pareto ·选择 AI 芯片工程 Flow：按任务、证据与成本比较

12.1 能力目标

- 画出规格、候选实现、工具执行与独立判定之间的数据和控制路径。
- 定义可复现的工具契约，区分失败、成功与结果未知。
- 建立独立的 golden reference，识别实现和测试共享错误的风险。
- 计算串行/并行检查的墙钟时间，并说明资源和依赖假设。
- 设计有版本、预算、职责和集成门槛的协作流程。

12.2 为什么需要：生成完成不等于工程完成

一个输出数值的性能模型可能把带宽单位误写成 byte/s；一个能编译的加法器可能回绕而规格要求饱和；一个通过的回归可能运行了旧提交。它们共同的问题是：产物与规格之间缺少明确的证据链。

工作流（workflow）预先确定工具和分支；**智能体（agent）**让模型在约束内选择下一步。确定的编译、报告解析和测试适合固定节点；故障定位、候选修复适合由模型提出方案。这个划分来自 Anthropic 的工程模式总结，属于实践建议，不是芯片性能基准。

可以把流程类比为 CPU 的控制状态机：规格和任务状态是控制寄存器，工具是执行单元，日志是可观测信号。类比的边界是：模型决策具有随机性，外部工具也可能超时或产生部分输出，因此不能把语言回答视为稳定的组合逻辑。

12.3 系统位置：谁提出改变，谁决定通过

完整路径为：**人确认规格 → 版本化契约 → AI 提出候选 → 工具适配层执行 → 独立检查器判定 → 证据归档 → 接受或修复。**模型读取日志并选择下一步，但不能把自己的“已完成”覆盖检查器的失败状态。

至少分开三类对象：① 规格与约束，例如输入形状、精度、复位、吞吐口径；② 候选实现，例如 RTL、模型代码、映射参数；③ 证据，例如测试用例、事件轨迹、原始计数器、工具报告。修改规格应产生单独差异和理由，不能为了通过测试悄悄改预期答案。

工具返回值除了 exit code，还应含 run_id、输入哈希、工具版本、完整命令、开始/结束时间、报告路径和完成状态。零退出码只表示工具按其约定结束；空测试集合、缺失报告、旧缓存仍须识别。

12.4 上下文不是全量粘贴：建立最小可复现输入

一次修复任务的上下文按优先级排列：规格版本和不可改变项 → 当前提交与可写范围 → 验收命令 → 最小失败输入 → 期望/实际差异 → 相关实现片段 → 已排除的假设。完整日志存文件，提示只放相关窗口与可追溯路径。

表 1: 表：上下文不是全量粘贴：建立最小可复现输入

工件	必须回答的问题
spec.json	位宽、单位、边界、吞吐/延迟定义是什么？哪些来自规范，哪些是估算？
candidate.patch	基于哪个提交？改了什么？是否越过职责边界？
run.json	运行哪套工具与配置？是否使用旧缓存？哪些步骤完成？
evidence.json	哪些检查通过？覆盖了什么？还不能证明什么？

架构模型尤其要记录参数出处：公开规范、微基准拟合、设计假设、占位值应分别标记。未知值保持未知，再做敏感性区间；AI 补出的一个 “合理常数” 不能升级为硬件事实。

12.5 状态变化：有上限的反馈环

建议显式记录状态：SPEC_READY → PLAN_READY → RUNNING → EVIDENCE_READY → ACCEPTED。已确认失败进入 REPAIR_PROPOSED，再运行；预算耗尽、规范矛盾或缺失关键资料进入 NEEDS_REVIEW。任务完成的条件是证据满足契约，不是对话结束。

先分类再修复：环境错误应修依赖；语法/类型错误应修接口；功能错误应定位语义；模型偏差应拆解计数与资源占用；PPA 不达标应审视结构和约束；结果未知则先查询执行状态。不同类别不能都用 “再让模型重写一次” 处理。

AutoChip 2025 v3 研究了编译/仿真反馈驱动的有限轮次 RTL 修复，但收益随所测模型和预算变化。它使用 VerilogEval V1，不能把结果直接套到 V2 或当前模型。教学流程借鉴其反馈结构，不承诺复现其收益。

12.6 约束：持久化、重试与幂等

检查点（checkpoint）保存流程状态；**幂等（idempotency）**指重复请求不会产生额外业务效果。二者不同。若提交 P&R 作业成功但回执丢失，直接重试可能重复占用资源，不能把超时当成 “没有执行”。LangGraph 故障恢复文档明确提醒：节点重试会重新执行，外部副作用没有天然的 exactly-once 保证。

做法是为同一次逻辑操作使用稳定 operation_id：先查询集群作业和产物，再选择接管已有执行或重新提交。每个 trial 使用独立目录；报告写临时文件，完整性校验后才标记完成；验收时核对输入哈希。耗时长的执行宜与结果解析拆开，使解析重试不必重新跑 EDA。

并发写同一个结果文件、旧分支覆盖新实现、重试重复提交，都是流程正确性问题，即使模型每次回答都正确也会发生。

12.7 独立判定：先确定答案从哪里来

参考判定器（oracle）可以是数学定义、独立实现、性质约束或经过校准的测量。它不必由另一个大模型担任。若生成器和测试器都误解同一句规格，两者一致也可能一起错。

RTL例：8 位无符号饱和加法要求 min(a+b,255)。对 200+100，正确输出 255，8 位截断输出 44。独立数学参考和边界用例能揭示错误；从错误 RTL 自动抄出的期望值不能。cocotb 负责连接 Python 与 HDL 仿真器，正确性仍来自你的参考与检查。边沿后立即读数还可能产生采样竞争：cocotb 2.0.0 timing model 区分 RisingEdge 事件与当前时间步稳定后的 ReadOnly。

架构模型例：先以手算验证单资源无冲突场景，再加共享端口、队列、依赖和背压。检查守恒：发出请求数 = 完成数 + 在途数；资源使用不超过容量；任务不能早于输入就绪。随后以独立微基准校准延迟和吞吐，留出未参与拟合的形状验证。Accel-Sim 作者框架展示了 trace、模型与硬件关联比较的工程分工；其模型和验证结果不自动适用于新芯片。

12.8 完整算例：并行节省了什么

以下均为教学假设，不是实测。一次候选生成 8 s，编译 4 s；编译后仿真 24 s、性质检查 36 s；汇总 3 s。两项检查互不依赖、有独立机器和许可证、无排队、均成功。

串行墙钟时间 T_serial=8+4+24+36+3=75 s。并行时间 T_parallel=8+4+max(24,36)+3=51 s。加速比 =75/51≈1.47。检查资源消耗仍为 24+36=60 资源秒；墙钟减少不表示总计算量或费用同比减少。

若失败后定位和修复需 10 s，并只重跑上述编译与两项检查，则追加 10+4+36+3=53 s，总计 104 s。若仅有一份共享许可证，24 和 36 不能重叠，51 s 的结论不成立。若存在缓存、失败概率、资源抢占，应另建模型，不能沿用这个理想公式。

别混淆四个时间：工程工具运行的秒、硬件模拟周期、候选硬件的工作频率、用户请求的端到端延迟。一个模拟器运行更快，未必意味着它模拟的芯片更快。

12.9 取舍与误解：何时增加 agent

先让一个受约束的流程稳定工作，再考虑并行。适合拆开的任务包括：同一冻结规格的独立测试设计、互不重叠的模块实现、不同参数 trial。每个工作者需要基准提交、可写文件、接口、产物格式和预算；集成者在合并后的提交上重新验证。

两个 agent 同时改同一文件会增加冲突；多个 agent 共享错误前提不会形成独立证据；让评审者只能读实现而不读规格，会偏向确认现有代码。多 agent 的价值来自任务结构和检查独立性。

VerilogEval 作者仓库说明版本、提示与采样设置的变化，因此“成功率提高”必须同时说明任务集和尝试预算。阅读完成只记学习进度；能独立画出状态图、复现一次失败并证明修复，才记能力证据。下一章用相同原则选择具体工程 Flow。

12.10 能力训练与完整答案

练习 1 ·示范

题目：示范：饱和加法器对 200+100 输出 44。把一次修复写成可验收记录。

补全步骤与提示：先固定规格和最小失败输入，再区分可改实现与不可改的预期值。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 记录 spec=v1、输入 200/100、期望 255、实际 44 和失败波形。怀疑先做 8 位截断；只改 DUT，用 9 位和判断是否超过 255。独立参考保持不变，补 0+0、255+0、255+1、连续输入与复位检查。对当前提交重跑全部回归，再保存工具版本、命令和结果。

自评量表

- 写出饱和而非回绕规格
- 给出最小可复现的期望/实际
- 明确不可偷偷改测试预期
- 有边界与时序回归证据

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 ·补全

题目：补全：作业提交回执丢失，应进入 ____ 状态，并通过 ____ 查询。若生成 5 s、编译 5 s、两项独立检查 20/30 s、汇总 2 s，理想并行总耗时为 ____。

补全步骤与提示：保留原 operation_id；先画依赖，再选择相加还是取最大值。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 进入 UNKNOWN；以稳定 operation_id 查询集群作业和产物，存在则接管，确认不存在才重提。总耗时 5+5+max(20,30)+2=42 s；假设独立资源、无排队和失败。

自评量表

- 识别未知结果而非简单失败
- 保留稳定操作 ID
- 先对账再重试
- 算出 42 s 并说明资源假设

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 •独立设计

题目：为 AI 辅助的双资源事件模拟器设计协作流程：一个 agent 写调度，另一个写验证。如何避免双方把同一个错误重叠假设写进实现和测试？

补全步骤与提示：交付状态图、工件表、3 个独立 oracle 场景、一次超时恢复和集成规则。题目自评，不使用不可靠的自动判分。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 冻结单/双端口、非抢占服务、依赖、单位和完成事件定义。调度 agent 只改实现；验证 agent 从冻结契约与手算轨迹设计测试。覆盖独立资源可重叠、共享端口不可重叠、消费者等待生产者；加入请求守恒和容量约束。各自固定基准提交，合并后重验；外部执行使用操作 ID 对账；设修复次数和耗时预算。

自评量表

- 规格明确资源/依赖/单位
- 验证有独立手算与不变量
- 权限、版本和集成责任清晰
- 包含未知恢复和停止条件

评分：满分 4 分；开放答案采用自评，不自动判定正确。

12.11 口述与追问

为什么有一条测试命令仍不足以构成可靠的 agent 验收？

参考回答：还要确认输入/工具版本、测试确实运行、预期值独立、覆盖边界、报告完整、执行的是当前提交。命令只是执行入口，不能定义全部证据。

- 如何发现测试集合为空？
- 如果 DUT 与测试都能被 agent 修改，如何控制？

远程仿真提交后网络断开，任务应标失败还是未知？

参考回答：标结果未知，使用稳定操作 ID 查询作业状态或产物；确认不存在才重提。持久化流程状态不保证外部提交恰好一次。

- 怎样让报告解析可以安全重试？
- 若集群不支持幂等键怎么办？

AI 写的事件模拟器与解析模型不一致，先相信哪个？

参考回答：先找两者假设和统计边界：依赖、端口争用、启动延迟、稳态吞吐、数据移动。用最小手算场景和独立测量定位；更详细的模型也可能有 bug。

- 哪个简单不变量能查出负队列？
- 如何分开拟合集与验证集？

12.12 快速检查

1. 提交远程 EDA 作业后超时，最合适的第一步是?

A. 立刻提交十份重试
B. 按失败统计并丢弃日志
C. 将状态标为未知，用 operation_id 查询已有执行
D. 把超时改成通过

答案：C：超时不能证明外部动作未发生。先对账，再接管或重提，避免重复副作用。

2. 上文 51 s 并行算例中，哪项变化直接破坏 max(24,36) 的假设?

A. 报告中增加版本字符串
B. 两项检查需要同一份不可共享许可证
C. 输出文件改名
D. 生成器回答更简短

答案：B：独占资源使两项检查不能重叠，时间模型要加入串行和排队。

3. 实现和测试都由同一错误规格生成并全部通过，能得出什么?

A. 规格已正确
B. 可直接流片
C. 在现有检查下两者一致，仍需独立验证规格含义
D. 可以忽略边界条件

答案：C：共享错误会造成相关失效；一致性证据不能替代规格正确性。

12.13 来源与阅读边界

- [src-anthropic-patterns] Building effective agents
- [src-langgraph-retry] LangGraph Fault tolerance
- [src-autochip-2025] Automatically Improving LLM-based Verilog Generation using EDA Tool Feedback
- [src-cocotb-2] cocotb 2.0.0 Documentation
- [src-cocotb-timing] The Timing Model - cocotb 2.0.0
- [src-verilogeval] NVlabs/VerilogEval author repository
- [src-accelsim] Accel-Sim author framework repository

13 选择 AI 芯片工程 Flow：按任务、证据与成本比较

模块 10 ·深入选学·建议 60 分钟

当前值得借鉴的是能把工程输入、工具反馈、独立验证和版本追溯连接起来的流程。本章围绕架构性能建模，再比较 SFU/HLS、RTL 修复、规格生成、知识检索与 PPA 探索；工具能力与研究证据分开评价。

先修：工程 Agent：状态、上下文与可验证交付 ·PPA 与候选筛选：证据、约束和 Pareto

横向关联：架构模型：先界定问题，再选择抽象层级 ·算子融合与 L1：数据搬运为什么支配结果 ·验证闭环：怎样证明 AI 写的实现符合规格

13.1 能力目标

- 按任务输入、交付物、判定器和失败方式选择工程 Flow。
- 区分确定性工具自动化、LLM候选生成与代理编排。
- 建立架构模型的手算、微基准、校准集和保留集验证链。
- 在相同精度、负载、资源和尝试预算下设计 PoC。
- 先检查功能与硬约束，再比较性能、能量和人工成本。

13.2 先定义好在哪里：不是一张工具排行榜

“做得好” 在本课程中表示：任务边界明确、输入输出可检查、错误能退出、证据可重放，并在相同预算下有实用价值。一个擅长生成 RTL 的工具不能直接与知识库问答平台比较；旧论文结果也不能替代当前版本的自有任务验证。

为每个候选方案填四项：输入契约是什么；交付物是什么；谁独立判断正确；最可能在哪种边界失败。再量化正确率、首次正确耗时、尾部耗时、总 token/计算资源和人工介入。先建立最小基线，再加模型或多 agent，才知道收益来自哪里。

证据标签：工具文档说明能力；作者论文说明特定实验；厂商网站说明产品主张；本章流程是工程设计建议；本章数字均为教学算例，不是这些工具的测量值。

13.3 系统位置：让不同工具承担不同职责

表 2: 表：系统位置：让不同工具承担不同职责

问题	建议组合与交付	独立证据
架构性能预测	规格 schema → AI 辅助解析/事件模型 → 轨迹与指标	手算、不变量、微基准、未拟合负载
算子映射与融合	结构化图/IR → 合法候选 → 容量/资源模型	数值等价、实际移动量、时间线
算法数据路径	golden → DSLX/XLS → RTL → EDA	误差预算、RTL 回归、同条件 STA/PPA
局部 RTL 修复	冻结规格 → 候选 patch → lint/仿真反馈	独立测试与性质、全量回归
寄存器/文档	原表 → 规范化 schema → 确定性生成	源单元格追溯、行为语义检查
工程知识问答	版本过滤 → 检索 → 引文与假设 → 复现	引文支持结论、实际工具验证
实现参数探索	固定约束 → ORFS trial → 可行性/Pareto	同工艺同阶段完整报告

这些是可组合的工程角色。自动化编译器、仿真器和搜索器不因为被 AI 调用就变成 AI 模型；模型解释报告也不改变报告本身的证据等级。

13.4 首要 Flow：让 AI 帮你建立可校准的架构模型

先以显式 schema 固定算子、精度、形状、存储层次、带宽单位、资源速率和依赖；再让 AI 实现解析估算与事件模型。架构初期可用分析模型快速扫描；怀疑端口争用、队列、背压或启动开销时，再增加相应事件。更细不是自动更准确，每新增状态都需要验证。

Timeloop/Accelergy 把工作负载、映射、架构与技术成本分开；v4 架构格式显式描述存储容量、位宽、读写/共享带宽和缓冲份数。它适合借鉴输入结构与映射分析，但不能假定任意自定义指令、银行冲突或融合执行都已被覆盖。MAESTRO 也是数据流/成本建模考察对象，选用前要核对支持算子与硬件假设。

验证顺序：① 无冲突单操作手算；② 依赖链与共享资源反例；③ 请求守恒和资源容量不变量；④ 小型独立微基准测参数；⑤ 冻结参数后用保留形状验证误差。用平均误差掩盖特定算子失真不可接受，应按算子、规模、访存模式分层报告。

模型输出应带证据等级和假设：这是估算，不是实测；遗漏了哪些资源；参数来自何处；对关键未知参数的敏感性有多大。AI 可以帮助补代码和分析残差，不能靠修改每个样本的常数“拟合正确”。

13.5 融合与映射：先保留语义，再比较资源

用中间表示（intermediate representation, IR）记录形状、索引、归约和依赖，能把“图上连在一起”变成可检查的计算结构。MLIR Transform 官方教程展示了结构化操作与融合的表达方式，某些融合可去掉临时缓冲。编译合法性与硬件收益仍是两个问题。

每个候选同时检查：计算结果/舍入是否符合要求；tile 的活跃工作集能否放下；双缓冲是否真的有两份容量；读写端口是否互相争用；融合是否增加重算或降低并行度。算子名相同不代表 ISA 分解相同，分解到不同执行资源后必须重新计算工作量和传输量。

教学反例：融合少写一个中间张量，但使每个 tile 的活跃数据超过本地容量；若发生 spill 或缩小 tile 导致更多重复读取，总时间可能增加。先画两条时间线，再比较总流量和关键资源，不能只统计少了几次 kernel launch。

13.6 HLS 与规格生成：把确认过的含义交给确定性工具

SFU等有明确数学定义的数据路径，可采用独立高精度参考 → 误差与位宽预算 → AI 辅助 DSLX → XLS 测试/IR/调度/RTL → RTL 回归 → 同工艺实现。XLS 官方说明将其定位为仍在演进的 HLS 工具链；调度文档中的内部延迟模型不是布线后 STA。任意 Python 并不能不经转换就成为可综合硬件。

Excel 规格到代码宜先提取规范化字段和源单元格地址，歧义交给规格责任人。对于控制/状态寄存器，PeakRDL 可从统一描述生成多种产物，避免多份手工同步；同时核对已知限制。AI 适合解释和发现歧义，确定性生成器适合重复产出。

反例：W1C（写 1 清零）被解析成普通 RW（读写），RTL、头文件和文档可能完全一致却全部错误。除结构一致性外，必须独立验证访问语义、复位值、保留位和跨时钟更新等实际行为。

13.7 RTL 修复与知识检索：反馈必须能反驳候选

局部 RTL 修复的基线是一轮候选加人工检查；增强方案是有限预算的 lint/编译 → 自检仿真 → 最小失败日志 → patch → 全回归。AutoChip 2025 提供这种 EDA 反馈结构的研究案例；与 VerilogEval V2 比较时须注意它使用的是 V1。测试通过不等于覆盖全部功能，不能把公开题答案泄漏当作泛化能力。

ChipNeMo 研究工程问答、EDA 脚本与 bug 总结，使用领域数据和检索；内部语料限制外部复现，旧模型比较不代表当前排名。Dify 检索节点提供元数据过滤、重排与上下文引用等组装部件。应用时按项目、文档版本、工具版本过滤，再要求回答区分原文事实和待验证假设。

“有引用”不等于引用支持结论；一个型号的手册也不能填补另一个型号的未知参数。知识流的终点应是最小复现或可追溯决策，而不是听起来可信的答案。

13.8 PPA 和商业平台：统一任务条件后再做选择

OpenROAD Flow Scripts 连接 RTL 到物理实现阶段；AutoTuner 提供配置搜索与指标采集。AI 可以提议候选或解释失败，但不能替工具编造 timing、面积或功耗。先固定功能、PDK/库、PVT、时钟/IO 约束、面积统计范围、流程阶段和预算，再筛除不可行点，最后比较 Pareto 前沿。

放宽时钟获得更小面积改变了问题。用默认活动率估计的动态功耗不能直接当成目标工作负载实测功耗；没有 VCD/SAIF 或其他活动依据时须明确其假设。生成 GDS 也不等于通过目标代工厂全部签核。

ChipAgents 可作为商业集成平台的 PoC 候选；公开产品页面属于厂商能力主张，本课程没有验证其兼容性、部署、报价或加速倍数。n8n evaluation 这类编排工具可用于派发、评测与报告，但执行成功不能替代 RTL 正确或模型校准。选择依据应是自有脱敏任务的可复现证据。

13.9 完整算例：统一预算的比较怎么报告

以下为教学数据，不指代任何工具。两条流程各运行同一 20 个独立任务，时限 120 min、相同工具与输入，统计全部失败和重试。A 成功 14 个，消耗 80,000 token；B 成功 16 个，消耗 120,000 token。

成功率 A=14/20=70%，B=16/20=80%。绝对提高 10 个百分点；相对提高 (80%-70%)/70%≈14.3%。每个成功任务分摊的总 token 为 A≈5,714，B=7,500；分子必须包含失败尝试，不能只数成功那一轮。

B 在这个样本中多完成两个任务，也消耗更多 token。未给输入/输出 token 划分和当时价格，不能算费用；token 也不是能耗。20 个样本不足以仅凭这两个百分比证明普遍优势，还要保留逐任务配对结果、重复运行、人工介入时间和失败类型。

架构模型的比较另加：同精度/形状/融合方案的预测误差、资源计数残差、保留集表现；实现性能的比较另加：同约束的时序和能量。把不同层次的数字塞进一个 “综合提升倍数” 会失去可解释性。

13.10 实施计划与迁移：从一个可证明的小闭环开始

三天路线：第 1 天冻结一个算子或小模块，写手算和独立参考；第 2 天搭建有版本日志的生成/执行/反馈闭环；第 3 天加入反例、未知状态恢复和同预算比较。完成标准是一条可复现证据链，而不是工具列表。

两周路线：第 1-2 天契约与基线；第 3-4 天模型或 RTL 的最小实现；第 5-7 天边界、资源冲突和保留集；第 8-10 天扩展融合、XLS 或 PPA 中的一个方向；第 11-12 天注入过期报告、丢回执、错误单位等故障；第 13-14 天统一预算比较、复查来源与复现说明。

用熟悉的 CPU/GPU 知识迁移：把新执行单元视为资源，把依赖视为发射条件，把本地存储视为受容量和端口约束的资源。但不要直接移植缓存一致性、warp 调度或乱序执行假设；只有目标架构明确存在的机制才能进入模型。未知参数先列为假设和敏感性范围。

13.11 能力训练与完整答案

练习 1 •示范

题目：示范：AI写的事件模型把共享读写端口当成两个独立端口，如何用最小实验发现?

补全步骤与提示：固定两个请求：读取服务 3 cycle，写入服务 2 cycle；均在 t=0 就绪，非抢占，端口只能服务一个请求，无其他延迟。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 独立手算：若读优先，读占 [0,3)，写占 [3,5)，makespan=5 cycle；写优先总时间仍为 5。错误模型会给 max(3,2)=3。检查每个时刻资源占用 ≤1，并核对完成数 =2。再将配置改成两个真正独立端口，3 cycle 才成立。这个反例验证建模语义，不是某芯片的测量。

自评量表

- 声明共享端口和非抢占
- 给出 5 cycle 手算轨迹
- 用占用不变量定位错误
- 用独立端口变式检验模型切换

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 •补全

题目：补全：同一 30 任务，A 成功 18 个共 90,000 token；B 成功 24 个共 150,000 token。求成功率、百分点差和每成功任务总 token，并指出尚不能得出的结论。

补全步骤与提示：使用全部任务作分母；失败尝试计入总成本；区分百分点、相对比例和费用。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 A=60%，B=80%，相差 20 个百分点，B 相对 A 提高约 33.3%。每成功任务 A=5,000 token，B=6,250 token。不能由此证明总体显著优势、实际价格或能耗，也不能忽略人工介入与任务配对。

自评量表

- 正确计算两成功率
- 区分 20 个百分点与 33.3%
- 包括失败成本的 token 分摊正确
- 指出统计和成本结论边界

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 ·独立设计

题目：为一个未知架构的矩阵计算 + 激活融合项目设计两条可比较 Flow：一条快速分析，一条带事件队列的模型。怎样证明后者值得额外成本？

补全步骤与提示：交付统一 schema、资源/依赖表、手算基线、校准集/保留集、误差与耗时口径、停止标准。自评开放题。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 两者共享形状/精度/映射/参数来源和统计起止。分析模型给出明确的理想重叠假设；事件模型加入需验证的端口、队列、依赖和启动延迟。先通过相同手算与守恒测试，再以独立微基准校准，保留冲突/不规则形状验证。记录分层误差、预测运行时间、维护成本；只有解释了基线无法解释且可重复的偏差，才证明新增复杂度有价值。

自评量表

- 统一输入和统计边界
- 事件机制有独立验证依据
- 校准与保留集分开
- 同时比较误差、运行和维护成本

评分：满分 4 分；开放答案采用自评，不自动判定正确。

13.12 口述与追问

为什么不能直接说哪个 AI 芯片工具最好?

参考回答：工具面向不同任务。应固定任务集、输入、oracle、预算和部署限制，比较正确率、人工成本、时间与证据完整性。官网主张、论文实验和本地复现必须分开。

- 如何防止挑选演示用例?
- 如何统计失败重试的成本?

为什么一个模型在拟合集上准确仍不够?

参考回答：它可能把未建模机制吸收到常数中。需要未参与校准的形状和访存模式验证，并用资源计数、时序和不变量解释残差。

- 怎样设计带宽和启动开销的独立微基准?
- 如何识别以峰值参数冒充有效带宽?

融合减少了外部写回，为什么仍可能更慢?

参考回答：融合可能增大活跃工作集、导致 spill 或重复读取、改变布局、增加同步或降低并行。必须在相同精度和任务范围内比较数据流与关键资源时间。

- 如何构造容量临界点反例?
- 为什么图节点减少不能当成加速证据?

13.13 快速检查

1. 同一 20 题中 14 题与 16 题通过，成功率绝对差为?

A. 2%
B. 10 个百分点
C. 14.3 个百分点
D. 50%

答案：B：14/20=70%，16/20=80%，相差 10 个百分点；14.3% 是相对于 70% 的相对提升。

2. XLS 内部调度估算满足时钟，能直接证明什么?

A. 目标芯片硅后频率达标
B. 任意工艺 PPA 都达标
C. 在所用调度模型与配置下得到相应安排，仍需后续实现验证
D. 默认活动率等于目标负载功耗

答案：C：调度延迟模型、物理 STA 和硅后测量属于不同层次，不能互相冒充。

3. 厂商演示某任务成功后，最合理的下一步是?

A. 把演示速度写成生产保证
B. 取消独立测试
C. 用自有脱敏任务固定条件做 PoC 并记录全部成本
D. 只统计成功案例

答案：C：演示说明一种可能能力；你的约束、工具、数据和任务分布需要单独验证。

13.14 来源与阅读边界

- [src-timeloop-overview] Timeloop/Accelergy Overview
- [src-timeloop-v4] Timeloop+Accelergy v4 Architecture
- [src-mlir-transform] MLIR Transform Tutorial - Chapter 0
- [src-maestro] MAESTRO Documentation
- [src-accelsim] Accel-Sim author framework repository
- [src-xls-overview] XLS: Accelerated HW Synthesis - Overview
- [src-xls-scheduling] XLS Pipeline Scheduling
- [src-orfs-flow] OpenROAD Flow Scripts - RTL-to-GDSII Flow
- [src-orfs-autotuner] OpenROAD Flow Scripts AutoTuner
- [src-peakrdl] PeakRDL Introduction
- [src-peakrdl-limits] PeakRDL-regblock Known Limitations
- [src-autochip-2025] Automatically Improving LLM-based Verilog Generation using EDA Tool Feedback
- [src-verilogeval] NVlabs/VerilogEval author repository
- [src-chipnemo] ChipNeMo: Domain-Adapted LLMs for Chip Design
- [src-dify-rag] Dify Classic Knowledge Retrieval
- [src-chipagents-vendor] ChipAgents - IC-star product overview
- [src-n8n-evals] Introducing Evaluations for AI workflows

14 综合项目：交付一个经得起复查的工程闭环

模块 11 ·必须掌握·建议 100 分钟

把视频中两条主线收敛成可以复现的最小项目。这里提供已经运行的 RTL 示例与模型实验，再给出扩展任务；没有把尚未执行的 XLS/物理实现写成完成成果。

先修：验证闭环：怎样证明 AI 写的实现符合规格 ·模型验证与校准：把 “看起来合理” 变成可检查 ·选择 AI 芯片工程 Flow：按任务、证据与成本比较

横向关联：先把问题写成可验收的规格 ·SFU 与 LUT：分清数学误差和硬件一致性 ·PPA 与候选筛选：证据、约束和 Pareto

14.1 能力目标

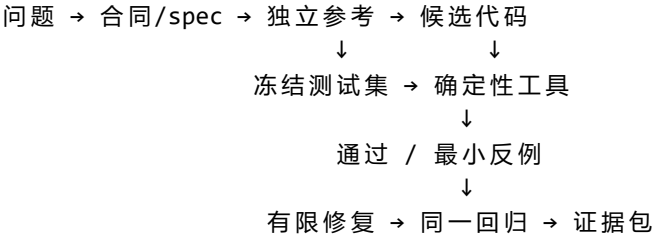
- 从工作负载写出验收合同和依赖图
- 运行确定性验证并保留失败反例
- 把模型假设、校准和保留集结果写进证据包
- 根据资源预算选择下一步并声明未完成能力

14.2 为什么需要：从理解到能力证据

读懂图表和独立交付不是同一能力。毕业要求不是背出工具名，而是别人拿到你的规格、代码、命令和版本，能重现 PASS，也能重现故意注入错误后的 FAIL，并且知道结果不能支持哪些结论。

默认背景是计算机体系结构与数字逻辑基础；若缺少 ready/valid 或浮点知识，先补相关章节。三天路线用于建立主线，两周路线用于完成项目；实际投入随基础和工具环境调整。

14.3 完整路径与交付对象



- spec：功能、输入域、接口、单位、目标与拒绝条件。
- implementation：源码、配置与哈希；AI 提案和工具决定分开。
- verification：参考来源、固定测试、种子、失败向量和判定。
- model：公式、变量、参数来源、缺失机制及适用范围。
- report：工具版本、运行命令、状态、差异和可复现产物。

14.4 示范项目 A：真实运行的饱和加法 RTL

规格是 8 位无符号 y=min(255,a+b)，一项弹性输出寄存器，valid/ready 握手，低有效异步复位。源码 examples/sat-add/sat_add8.sv 与独立 testbench 一同交付。组合输入 65,536 对逐一检查；随后加入定向输入和 400 个确定性随机握手/复位周期。

工具顺序为 Verible lint → Verilator lint → Icarus 编译与仿真 → 故意错误的变异测试。错误变异将饱和改成截断回绕，必须被独立参考拒绝。该结果只证明所执行用例的行为，不涵盖任意长时序序列或形式等价。

```
node tools/test-rtl.cjs
node --test tests/models.test.cjs
npm test
npm run build
```

实际检查数量、版本和运行证据见下载区 validation-report.json。网页 oracle 是另一项小型教学实验。未执行 XLS 全链路、OpenROAD PPA 或流片签核。

14.5 补全项目 B：可校准的性能模型

采用合成系统：512 Gop、64 GB外部访问、128 Gop/s 峰值、32 GB/s 峰值带宽、效率 η=0.5、启动 5 ms。按所选效率与重叠假设估算 4.005 s，串行则为 8.005 s。把数据写成配置而非散落代码。

下一步为两种容量和三种 tile 计划记录实际流量，检查共享带宽；加入跨算子依赖后画事件与资源占用。只有取得可信参考才能做校准。没有参考时交付敏感度与假设范围，不编造“误差低于 5%”。

14.6 独立项目：SFU 或融合算子二选一

- SFU：选有界函数，明确输入域、精度、特殊值和接口；先验数学近似，再验有限精度实现，再做固定约束的候选比较。
- 建模：选 softmax 或 gather→GEMM，明确 shape/dtype/映射，分解算子与数据流；设计容量溢出和共享资源冲突反例。
- 两条路线都固定预算、停止条件和保留集。AI可以提议更改，不能静默修改 oracle、阈值或基线。

14.7 项目验收量表：六项各 0-2 分

- 合同完整：0= 含糊；1= 有功能和目标；2= 边界、单位、接口和拒绝条件可执行。
- 机制解释：0= 工具罗列；1= 能讲路径；2= 能画数据/控制/状态并解释失败。
- 定量正确：0= 混淆单位；1= 基本计算对；2= 假设、统计范围、上下界一致。
- 验证独立：0= 复制实现；1= 独立用例；2= 边界、背压和变异证据齐全。
- 迁移能力：0= 只会示范；1= 能改参数；2= 处理陌生形状并指出边界。
- 可复现性：0= 只有结论；1= 有代码；2= 配置、版本、命令、日志、失败均可追溯。

建议至少 10/12 分，且合同、定量、独立验证三项不能为 0。这是教学自评，不是行业资格认证；与同伴交换证据包可发现遗漏。

14.8 失败案例与取舍

DUT与参考复制同一错误表达式，随机一万次也可能全绿；每个 shape 单独拟合效率，模型可能没有预测能力；功耗下降却改了频率和活动率，也不能证明同条件优化。都要回到合同与独立证据。

资源有限时先验证最能改变决策的假设：在整网长仿真前测一次共享端口冲突或 SFU 吞吐。没有可行候选时，交付失败范围与下一项最有信息量的实验也有价值。

14.9 复习与迁移

第 1 天不用资料画流程；第 3 天换位宽或 dtype；第 7 天加入背压或容量限制；第 14 天换陌生算子。在学习日志中分别记录阅读、自评证据与复习日期；开放答案按本手册量表自评。

14.10 能力训练与完整答案

练习 1 ·补全步骤

题目：饱和加法变成回绕加法后，预计什么反例会失败？

补全步骤与提示：选 a+b>255 的输入，分别计算两种语义。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 例如 255+1：规格 255，错误实现 0。保留输入、期望、实际、日志，修复后重跑完整回归。

自评量表

- 反例有效
- 结果正确
- 保留证据
- 完整回归

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 2 ·调试题

题目：模型在三个形状上很准，如何判断能否用于新架构？

补全步骤与提示：检查校准数据、参数自由度和资源覆盖。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 确认三个形状是否参与拟合；冻结参数后用新 shape、容量和分布验证；逐资源检查瓶颈、不变量和缺失机制。未验证范围不能外推为准确。

自评量表

- 区分拟合验证
- 独立保留集
- 机制检查
- 外推边界

评分：满分 4 分；开放答案采用自评，不自动判定正确。

练习 3 ·独立项目

题目：交付 SFU 或融合算子的证据包，并按六项量表自评。

补全步骤与提示：从合同开始，保留一个被测试拒绝的候选。

先独立写下假设、过程、结果和验证方法，再核对下面的答案。

参考答案 包括 spec、独立数学/时序参考、源码配置版本、测试报告、反例、模型边界、选择理由、重现命令和下一步。不强求没有条件执行的工具，但必须明示缺项。

自评量表

- 合同机制
- 独立验证
- 可重现反例
- 边界迁移

评分：满分 4 分；开放答案采用自评，不自动判定正确。

14.11 口述与追问

如何证明 AI flow 没有只是在自证正确?

参考回答: 冻结独立规格与 oracle，保留未知测试，执行负例/变异测试，保存运行配置、版本和判定。AI 提案可以变，接受标准需受控。

- 变异测试没失败怎么办?
- 参考也有缺陷怎么办?

14.12 快速检查

1. 没有可信参考时，性能模型最合适的交付是什么?

A. 宣称误差小于 5%
B. 报告假设、敏感度和待校准项
C. 把峰值称为实测
D. 删去失败形状

答案: B：未知校准误差不能由自洽公式或图表替代。

14.13 来源与阅读边界

- [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）
- [src-verilator-overview] Verilator Overview
- [src-cocotb-2] cocotb 2.0.0 Documentation
- [src-openai-repair] OpenAI - Build iterative repair loops with Codex
- [src-timeloop-v4] Timeloop+Accelergy v4 Architecture

15 工程 Flow 比较

15.1 架构性能模型：实现-校准-保留集

问题：用 AI 加快算子/资源性能模型的实现，同时避免漂亮图表掩盖错误假设。

基线：仅按峰值算力和外部带宽计算理想时间。

机制与步骤

1. 冻结精度、形状、映射、容量/端口/资源速率及参数来源。
2. AI 辅助实现解析模型；以独立手算与不变量验算。
3. 为确实需要表达的依赖、队列、冲突加入事件。
4. 用独立微基准校准；冻结参数后跑保留形状。
5. 输出事件轨迹、误差分层、版本和未覆盖机制。

证据：工程建议；Timeloop/Accelergy提供架构/映射建模结构，MLIR 提供结构化语义，Accel-Sim 展示 trace 与硬件关联验证；不声称已复现其性能。

成本：需要规格整理、单位检查、校准数据、模型维护和回归；详细模型可能显著增加运行时间。

适用条件：能明确关键资源和验证数据来源，且问题适合相应工具的建模范围。

反例：每个测试形状都单独调常数，拟合集很好但新形状失真；把共享读写端口当两个独立端口。

验证办法：先测单资源/依赖/争用反例，再用保留集按算子和形状报告误差；独立核对流量守恒与资源占用。

[src-timeloop-overview] Timeloop/Accelergy Overview ·[src-timeloop-v4] Timeloop+Accelergy v4 Architecture · [src-mlir-transform] MLIR Transform Tutorial - Chapter 0 ·[src-accelsim] Accel-Sim author framework repository ·[src-maestro] MAESTRO Documentation

15.2 算法与 SFU：golden-DSLX-XLS-RTL

问题：把明确的数值算法实现为可验证硬件数据路径。

基线：Python 参考模型与手写 RTL 分别维护。

机制与步骤

1. 冻结数学定义、输入范围、精度和误差预算。
2. 独立参考与边界向量先建立，AI辅助位宽明确的 DSLX。
3. 运行语言测试、IR 转换/优化和调度/代码生成。
4. 独立 RTL 回归，核对延迟、启动间隔和接口。
5. 固定工艺与约束进行综合/物理实现并保存报告。

证据：官方工具功能支持上述编译链；调度估计与后续实现是不同证据，本课程不报告测得的工具加速。

成本：版本/环境维护、数值误差分析、流水寄存器与接口适配。

适用条件：算法可表达为可综合结构，位宽、状态与吞吐需求明确。

反例：把无限精度 Python 直接当硬件语义；把 unit delay 模型时钟当硅后频率。

验证办法：高精度参考、极值/分段边界、RTL 事务时序、同条件 STA/PPA 分别检查。

[src-xls-overview] XLS: Accelerated HW Synthesis - Overview ·[src-xls-quickstart] XLS Tools Quick Start ·[src-xls-dslx] XLS DSLX Reference ·[src-xls-scheduling] XLS Pipeline Scheduling ·[src-xls-codegen] XLS Codegen Options · [src-orfs-flow] OpenROAD Flow Scripts - RTL-to-GDSII Flow

15.3 RTL 修复：有限反馈与独立验收

问题：减少一轮生成后大量人工定位，保留可复现的失败反馈。

基线：一次生成、人工查看波形、自由修改。

机制与步骤

1. 冻结规格、独立测试与验收命令。
2. 基于固定提交生成最小 patch。
3. 运行 lint/编译和自检仿真，分类错误。
4. 提供最小输入、期望/实际、事务时间与相关日志，限额修复。
5. 在当前提交重跑全回归与需要的性质检查。

证据：AutoChip 2025 展示 EDA 反馈结构但收益依模型而异；论文使用 VerilogEval V1，与 V2 设置分开。

成本：生成/执行预算、失败归类、独立 oracle 和回归运行。

适用条件：失败可运行重现，工具支持语言子集，规格不是边修边改。

反例：模型同时修改 DUT 和预期值掩盖失败；只优化公开基准题；无限次重试后只汇报最后成功。

验证办法：固定任务/提示/尝试预算，统计全部失败成本；审查规格和测试改动，保存当前提交的结果。

[src-autochip-2025] Automatically Improving LLM-based Verilog Generation using EDA Tool Feedback ·[src-verilogeval] NVlabs/VerilogEval author repository ·[src-cocotb-2] cocotb 2.0.0 Documentation ·[src-cocotb-timing] The Timing Model - cocotb 2.0.0 ·[src-verilator-overview] Verilator Overview

15.4 规格到产物：schema 与确定性生成

问题：减少 Excel/文档里的同一字段被多次解释并复制到代码。

基线：手工同步规格、RTL、头文件和文档。

机制与步骤

1. 解析字段并保留原文/单元格定位。
2. 规范化单位、复位、访问类型和空值语义。
3. 列出歧义，由规格责任人确认，禁止猜测填充。
4. 适用时转成 SystemRDL，锁定 PeakRDL 插件版本后生成。
5. 检查结构一致性，并独立验证行为语义。

证据：PeakRDL 提供多产物生成；具体能力受插件及已知限制约束。AI 解析部分是建议流程，不是官方准确率实验。

成本：schema 维护、异常格式处理、规格审查和行为测试。

适用条件：有明确字段语义和版本责任人，可将歧义暴露而不是静默吞掉。

反例：W1C 被解析成 RW，生成的所有文件一致却都错；别名/对齐规则超出所用插件支持范围。

验证办法：源字段到产物双向追溯、复位/访问行为测试、异常表格和重复地址检查。

[src-peakrdl] PeakRDL Introduction ·[src-peakrdl-limits] PeakRDL-regblock Known Limitations ·[src-anthropic-patterns] Building effective agents

15.5 工程 RAG：版本化证据到最小复现

问题：回答工具、架构与故障问题时混用文档版本或凭记忆补参数。

基线：搜索全文后人工归纳，没有答案来源与适用版本。

机制与步骤

1. 按项目、权限、文档版本和日期维护语料。
2. 用元数据过滤检索，再重排相关片段。
3. 输出支持结论的引文，并单列推断与未知。
4. 用最小工具实验或规范条款验证关键假设。
5. 归档根因、复现步骤和适用范围。

证据：ChipNeMo 提供领域知识适配的作者研究案例但内部语料限制复现；Dify 提供可搭建检索组件，不保证答案正确。

成本：语料清洗、版本/权限维护、召回与引用支持性评测。

适用条件：资料可合法访问，问题能由相关资料或实验支持，允许明确回答未知。

反例：引用同名不同版本寄存器；引用只提到术语却不支持答案；将私有厂商实验结论当作自己的复现。

验证办法：评测检索召回、版本准确性、引文支持性、未知问题处理，并以实际复现验证调试结论。

[src-chipnemo] ChipNeMo: Domain-Adapted LLMs for Chip Design ·[src-dify-rag] Dify Classic Knowledge Retrieval

15.6 实现探索：同条件搜索与可行性过滤

问题：有多个物理实现候选但报告条件不同，难以判断改进。

基线：手动改参数，只保存看起来最好的截图。

机制与步骤

1. 冻结功能、工艺/库/PVT、时钟 IO 约束和统计阶段。
2. 定义有限搜索空间、每次运行目录与总预算。
3. 运行 ORFS 候选，记录全部 trial 的配置、失败和指标。
4. 先去除功能/时序等硬约束不满足者，再找 Pareto 候选。
5. 重复关键候选，检查完整报告和工具版本。

证据：ORFS 与 AutoTuner 提供自动流程和参数探索能力；自动搜索不等于 LLM，不保证全局最优。

成本：机器/许可证、运行耗时、工艺资料与分析维护；能量结论需匹配活动假设。

适用条件：可合法使用工艺数据，固定评价边界，并能检测不完整和过期报告。

反例：放宽时钟后报告面积改善；把综合面积与布线面积混比；默认活动率当真实负载功耗。

验证办法：同条件重跑、核对输入哈希、报告阶段、可行约束、活动来源及失败 trial 统计。

[src-orfs-flow] OpenROAD Flow Scripts - RTL-to-GDSII Flow ·[src-orfs-tutorial] OpenROAD Flow Scripts Tutorial · [src-orfs-autotuner] OpenROAD Flow Scripts AutoTuner

15.7 团队 Agent：职责、幂等与证据门槛

问题：多人或多个 agent 协作时，版本、外部作业和验收状态容易不一致。

基线：共享聊天记录，任务完成靠口头说明。

机制与步骤

1. 建立任务 manifest、基准提交、可写范围与预算。
2. 冻结接口后分配独立任务，明确集成责任。
3. 持久化状态；外部执行使用稳定 ID 查询与对账。
4. 将报告解析、评测和归档做成可重复节点。

5. 在合并后的版本上验证，再由责任人接受。

证据：Anthropic 提供编排模式；LangGraph 说明重试与外部副作用边界；n8n 提供评测编排。商业 ChipAgents 仅列 PoC 对象，未独立验证。

成本：状态/接口设计、集成回归和可观测性；增加 agent 可能增加协调成本。

适用条件：任务可分解且有可检查的交付契约，外部工具行为可以查询。

反例：两个 agent 同时改同一文件；超时重复提交昂贵作业；把执行完成当成工程正确。

验证办法：故障注入丢回执、旧报告、过期 patch；验证可恢复且不重复副作用；记录全部人工介入。

[src-anthropic-patterns] Building effective agents ·[src-langgraph-retry] LangGraph Fault tolerance ·[src-n8n-evals] Introducing Evaluations for AI workflows ·[src-chipagents-vendor] ChipAgents - IC-star product overview

16 实验模型与边界

以下提供离线预测、手算和边界测试任务。在线界面仅是这些模型的配套演示，不是本 PDF 内置功能。运行实验前先写预测，随后用公开示例代码或网站核对。

16.1 独立参考：为什么全绿也可能全错

模型：8位无符号饱和加法的离散输入对照

假设：只检查组合算术，输入域为 0...255；三种候选实现与两种参考模型均明确展示

限制：不是完整 RTL 或时序仿真；完整例子另有自检仿真证据。

先预测：仅普通输入能检测溢出错误吗?

纸面任务：分别选择普通输入、边界输入、复制参考，解释测试结论变化。

迁移变式：把规格改成有符号饱和加法，旧测试计划还充分吗?

验证闭环：怎样证明 AI 写的实现符合规格

16.2 LUT 精度：分段、位宽与饱和

模型：分段线性插值 + 有符号定点输出量化

假设：中间计算 JS 双精度；4097 均匀采样点；相对误差仅统计 |f|≥0.01；系数不量化，最近舍入

限制：不是视频的二次 FP32/FMA 实现，不预测真实面积；节点存储只是代理。

先预测：段数翻倍后误差是否总是减为四分之一?

纸面任务：先改 N，再单独改 F 和 B；指出当前主误差来源。

迁移变式：固定 B=8，增加 F 为何可能让 exp 更差?

SFU 与 LUT：分清数学误差和硬件一致性

16.3 流水线与背压：看见每拍资源占用

模型：离散时钟边沿、每级 1 笔的弹性流水线

假设：12 个按间隔到达的事务；每级 1cycle，组合 ready 传播理想化；入口等待队列不设容量上限；边沿 0 接受；D 级首结果在边沿 D；物理历时与半开观察窗口完成率分别记录

限制：不是 XLS 调度仿真，不包含真实 ready 路径时序、CDC 或多周期算子。

先预测：D 从 4 增到 8，稳态吞吐会翻倍吗?

纸面任务：改级数与下游停顿，逐拍检查守恒、占用和输出顺序。

迁移变式：下游永不 ready 时，如何定义未完成和超时?

XLS 与流水线：从功能图到带时序的实现

16.4 架构瓶颈：算力、带宽与重叠

模型：计算/传输时间估计 + 可选理想重叠 + 启动常数

假设：GB/Gop 为十进制单位；计算模型假设持续达到所设计算速率；传输速率为峰值带宽 × 假设效率 η；固定总工作量，不包含 bank 竞争；所算时间是模型估计，不是无条件硬件下界

限制：教学解析模型，不是目标芯片或真实工作负载预测。

先预测：默认配置算力翻倍，端到端时间能减半吗?

纸面任务：分别改变算力、流量和利用率，观察瓶颈迁移。

迁移变式：如何设计实验区分串行传输与真正双缓冲重叠？

架构模型：先界定问题，再选择抽象层级

16.5 PPA 筛选：可行域与 Pareto 前沿

模型：预设教学候选的约束筛选与支配判定

假设：所有候选假定同功能、同工艺、同活动率；误差门槛先筛选，再比较面积/延迟/功耗

限制：全部数值为合成数据，不运行 EDA，不预测实际 PPA。

先预测：最快的候选是否一定是最佳？

纸面任务：收紧时序与误差约束，观察候选何时变为不可行。

迁移变式：没有可行候选时，应修改规格、扩展搜索还是报告失败？说明证据。

PPA 与候选筛选：证据、约束和 Pareto

17 术语索引

17.1 Specification ·规格

规定输入、输出、约束和验收，不只是功能口号。

先把问题写成可验收的规格

17.2 Oracle / Golden model ·独立参考模型

从规格推导预期结果；不能照抄被测实现的错误。

验证闭环：怎样证明 AI 写的实现符合规格

17.3 SFU ·特殊函数单元

加速 exp、sin、sigmoid 等函数的专用数据通路。

SFU 与 LUT：分清数学误差和硬件一致性

17.4 LUT ·查找表

用离线保存的节点或系数减少在线计算。

SFU 与 LUT：分清数学误差和硬件一致性

17.5 FMA ·融合乘加

a×b+c 在最终结果处统一舍入，区别于两次独立运算。

SFU 与 LUT：分清数学误差和硬件一致性

17.6 DSLX / XLS IR ·领域语言 / 中间表示

XLS 的高层描述与优化/调度所用表示；不是 Excel 文件格式。

XLS 与流水线：从功能图到带时序的实现

17.7 Latency / II ·延迟 / 启动间隔

前者是单笔完成等待，后者是连续接收的最小间隔。

XLS 与流水线：从功能图到带时序的实现

17.8 Backpressure ·背压

下游未就绪时阻止上游覆盖在途事务。

XLS 与流水线：从功能图到带时序的实现

17.9 Roofline ·屋顶线模型

用计算上界与数据传输上界定位瓶颈的解析模型。

架构模型：先界定问题，再选择抽象层级

17.10 Operational intensity ·运算强度

同一统计范围的总运算量除以某层级访问字节，单位 op/byte。

架构模型：先界定问题，再选择抽象层级

17.11 Mapping / Tiling ·映射 / 分块

把计算与数据划分到硬件资源和时空位置。

算子融合与 L1：数据搬运为什么支配结果

17.12 Working set ·工作集

某时刻必须同时保留的数据，不等于所有张量简单相加。

算子融合与 L1：数据搬运为什么支配结果

17.13 Calibration / Validation ·校准 / 有效性验证

前者选参数，后者在独立任务检查模型适用性。

模型验证与校准：把“看起来合理”变成可检查

17.14 PPA / STA ·功耗性能面积 / 静态时序分析

不同约束和证据层级，不能合为一个未经定义的分数。

PPA 与候选筛选：证据、约束和 Pareto

17.15 Pareto frontier ·帕累托前沿

可行候选中不存在另一个全面不差且至少一项更优的集合。

PPA 与候选筛选：证据、约束和 Pareto

17.16 Workflow / Agent ·固定工作流 / 动态智能体

前者规定控制路径；后者可动态选择动作。两者可混合。

工程 Agent：状态、上下文与可验证交付

17.17 Idempotency ·幂等性

同一逻辑操作重复执行不会造成额外副作用，须由具体工具协议保证。

工程 Agent：状态、上下文与可验证交付

18 完整来源索引

原规范、论文、代码和厂商文档版权属于原作者。这里提供原创教学整理和原始来源链接，不镜像全文。链接需联网访问；文档的适用版本、阅读范围和证据限制与正文同等重要。

18.1 [video] 用户提供的 2026-11-08 研讨录屏（脱敏归纳）

类型与版本：录屏陈述与演示；2026；01:23:45.40，录屏从会议中途开始
支持的证据：讨论与演示记录，不是独立复现实验。原视频、人物截图、逐字稿不随公开站发布。
阅读范围：完整音轨与关键画面；时间戳相对录屏起点
边界：原始录制从介绍中段开始；两遍本地转写结合画面复核，不保证逐字无误。主要技术发言到约 79:28，后段为摘要画面及短音频事件；仅公开匿名技术归纳，未独立复现视频 PPA/架构结果。
访问日期：2026-11-08
本手册录屏归纳索引

18.2 [src-xls-overview] XLS: Accelerated HW Synthesis - Overview

类型与版本：官方工具文档；2026；滚动文档，访问快照 2026-11-08
支持的证据：功能说明：高层描述经 XLS 产生可综合 Verilog/SystemVerilog；区分纯函数与有状态 proc。
阅读范围：What is XLS?；State of the Project
边界：公开，已读取
访问日期：2026-11-08
https://google.github.io/xls/

18.3 [src-xls-quickstart] XLS Tools Quick Start

类型与版本：官方教程；2026；滚动文档，访问快照 2026-11-08
支持的证据：工具链路径：DSLX 解释器测试 → ir_converter_main → opt_main → codegen_main。
阅读范围：Interpreting the DSL file；DSL to IR conversion；IR optimization；Verilog RTL generation
边界：公开，已读取
访问日期：2026-11-08
https://google.github.io/xls/tools_quick_start/

18.4 [src-xls-dslx] XLS DSLX Reference

类型与版本：官方语言参考；2026；滚动文档，访问快照 2026-11-08
支持的证据：语言契约：显式位宽与符号、类型转换、函数测试和进程状态。
阅读范围：Types；Bit Type；Type Casting；Testing and Debugging；Procs
边界：公开，已读取所列部分
访问日期：2026-11-08
https://google.github.io/xls/dslx_reference/

18.5 [src-xls-scheduling] XLS Pipeline Scheduling

类型与版本：官方机制文档；2026；滚动文档，访问快照 2026-11-08
支持的证据：调度根据依赖与内部延迟模型分配流水级，在时钟周期、级数和寄存器开销之间取舍。
阅读范围：Scheduling process；effective clock period；minimize pipeline registers
边界：公开，已读取
访问日期：2026-11-08
https://google.github.io/xls/scheduling/

18.6 [src-xls-codegen] XLS Codegen Options

类型与版本：官方参数参考；2026；滚动文档，访问快照 2026-11-08
支持的证据：generator、delay_model、clock_period_ps、pipeline_stages 为独立配置项。
阅读范围：Pipelining and Scheduling Options；输出 schedule 路径
边界：公开，已检索并读取相应条目
访问日期：2026-11-08
https://google.github.io/xls/codegen_options/

18.7 [src-orfs-flow] OpenROAD Flow Scripts - RTL-to-GDSII Flow

类型与版本：官方工具文档；2026；latest 文档，访问快照 2026-11-08
支持的证据：实现流程：Yosys 综合 → Floorplan → Placement → CTS → Routing → Finishing。输入包含 RTL、SDC、lib、LEF。
阅读范围：OpenROAD Flow 阶段图；Tool Installation 的版本限制
边界：公开，已读取
访问日期：2026-11-08
https://openroad-flow-scripts.readthedocs.io/en/latest/mainREADME.html

18.8 [src-orfs-tutorial] OpenROAD Flow Scripts Tutorial

类型与版本：官方实验教程；2026；latest 文档，访问快照 2026-11-08
支持的证据：教程以 ibex、sky130hd 演示设计配置、SDC 与流程输出。
阅读范围：Getting Started；Configuring the Design；Running the Automated RTL-to-GDS Flow
边界：公开，已读取所列部分
访问日期：2026-11-08
https://openroad-flow-scripts.readthedocs.io/en/latest/tutorials/FlowTutorial.html

18.9 [src-cocotb-2] cocotb 2.0.0 Documentation

类型与版本：官方框架文档；2026；2.0.0
支持的证据：Python 协程通过仿真器接口驱动/观察 HDL DUT；框架需要独立 HDL 仿真器。
阅读范围：What is cocotb；How does cocotb work
边界：公开，已读取
访问日期：2026-11-08
https://docs.cocotb.org/en/v2.0.0/

18.10 [src-cocotb-timing] The Timing Model - cocotb 2.0.0

类型与版本：官方语义文档；2026；2.0.0
支持的证据：事件采样语义：边沿触发发生时，下游 HDL 未必完成计算；ReadOnly 等到当前时间步稳定且不可写。
阅读范围：Values Change；Values Settle；End of Time Step；RisingEdge/ReadOnly
边界：公开，已读取
访问日期：2026-11-08
https://docs.cocotb.org/en/v2.0.0/timing_model.html

18.11 [src-verilator-overview] Verilator Overview

类型与版本：官方工具文档；2026；所读文档页标识 5.054
支持的证据：将 HDL 编译成 C++/SystemC 模型，可执行 lint，并按选项生成波形与覆盖数据。
阅读范围：Overview 使用步骤
边界：公开，已读取
访问日期：2026-11-08
https://verilator.org/guide/latest/overview.html

18.12 [src-verilogeval] NVlabs/VerilogEval author repository

类型与版本：作者代码与基准说明；2026；main/VerilogEval V2；另保留 release/1.0.0
支持的证据：V2 增加 spec-to-RTL、改写提示和错误分类；README 明确与 V1 的差别。
阅读范围：Overview；Setup Linux Environment；Usage
边界：公开，已读取 README
访问日期：2026-11-08
https://github.com/NVlabs/verilog-eval

18.13 [src-autochip-2025] Automatically Improving LLM-based Verilog Generation using EDA Tool Feedback

类型与版本：原始研究论文；2025；arXiv 2411.11856v3，2025-03-04
支持的证据：作者实验：生成候选 RTL → 编译/仿真 → 把失败反馈用于有限轮次修复。实验使用 VerilogEval V1。
阅读范围：§3；§4.1-4.3；§5.1-5.3；Table 1-2；Figure 9-10
边界：公开全文，已读取所列部分
访问日期：2026-11-08
https://arxiv.org/html/2411.11856v3

18.14 [src-chipnemo] ChipNeMo: Domain-Adapted LLMs for Chip Design

类型与版本：原始研究论文；2024；arXiv 2311.00176v5，2024-04-04
支持的证据：作者实验覆盖工程问答、EDA 脚本与 bug 总结，结合领域适配与检索。
阅读范围：§2.4；§3.5-3.7；Appendix A.3-A.5；Tables 3-5, 7-8
边界：公开全文；内部训练数据不可公开完整复现
访问日期：2026-11-08
https://arxiv.org/html/2311.00176v5

18.15 [src-chipagents-vendor] ChipAgents - IC-star product overview

类型与版本：厂商产品说明；2026；网页访问快照 2026-11-08
支持的证据：厂商声称提供半导体设计与验证的 agentic 平台，覆盖数字与模拟相关工作。
阅读范围：公开首页和产品定位
边界：公开首页；完整产品需要联系/账号
访问日期：2026-11-08
https://chipagents.ai/

18.16 [src-anthropic-patterns] Building effective agents

类型与版本：厂商工程实践；2024；2024-12-19 原文；所读页面有后续更新提示
支持的证据：工程模式：固定控制路径适合可分解任务；动态 agent 适合路径需探索的任务。
阅读范围：Workflow/agent 定义；chaining；parallelization；orchestrator-workers；evaluator-optimizer
边界：公开，已读取
访问日期：2026-11-08
https://www.anthropic.com/research/building-effective-agents

18.17 [src-langgraph-retry] LangGraph Fault tolerance

类型与版本：官方运行时文档；2026；Python 文档；节点超时与 error handler 要求 >=1.2
支持的证据：失败重试从节点函数开始；外部副作用不因此具有 exactly-once 保证。
阅读范围：Retries；Retry boundaries and idempotency；missing acknowledgement；Timeouts；Limitations
边界：公开，已读取
访问日期：2026-11-08
https://docs.langchain.com/oss/python/langgraph/fault-tolerance

18.18 [src-n8n-evals] Introducing Evaluations for AI workflows

类型与版本：官方功能说明；2025；2025-06-06；首发要求 n8n >=1.95.1
支持的证据：可独立于生产触发器运行评测输入与指标，结合代码指标和定性评审。
阅读范围：What are Evaluations；getting the most from AI Evaluations
边界：公开，已读取
访问日期：2026-11-08
https://blog.n8n.io/introducing-evaluations-for-ai-workflows/

18.19 [src-dify-rag] Dify Classic Knowledge Retrieval

类型与版本：官方产品文档；2026；页面标记 2026-10-28 更新
支持的证据：检索 → 重排/阈值 → 带元数据的片段 → LLM 上下文/引用。
阅读范围：Configure Node；node-level retrieval；metadata filtering；output；use with LLM
边界：公开，已读取
访问日期：2026-11-08
https://docs.dify.ai/en/cloud/use-dify/nodes/knowledge-retrieval

18.20 [src-openai-repair] OpenAI - Build iterative repair loops with Codex

类型与版本：官方工程示例；2026；2026-05-11
支持的证据：以可执行验证驱动有限迭代的通用工程范式；原例为文档修复。
阅读范围：Review / Repair / Validate 闭环与结构化反馈
边界：迁移到芯片工程是本课程设计，不是 OpenAI 对 RTL 正确性或 PPA 的实验结论。
访问日期：2026-11-08
https://developers.openai.com/cookbook/examples/codex/build_iterative_repair_loops_with_codex

18.21 [src-xls-float] XLS Floating-point routines

类型与版本：官方实现语义；2026；2026-11-08 滚动文档
支持的证据：FMA 单次最终舍入与库中浮点操作的特殊值/舍入边界。
阅读范围：APFloat；add/sub；mul；fma；Testing
边界：不把当前文档行为未经核对地归到视频中的旧 commit；不声称完整 IEEE 754 异常标志支持。
访问日期：2026-11-08
https://google.github.io/xls/floating_point/

18.22 [src-roofline] Roofline: An Insightful Visual Performance Model for Multicore Architectures

类型与版本：原始论文；2009；Williams / Waterman / Patterson，CACM April 2009，DOI 10.1145/1498765.1498785
支持的证据：用于解释为什么增加算力不一定降低运行时间。
阅读范围：运算强度、带宽与计算性能上界的基本模型
边界：公开收录页；直连可能触发 JavaScript 机器人验证。出版信息可在作者机构目录核对。
访问日期：2026-11-08
https://escholarship.org/uc/item/5tz795vq

18.23 [src-timeloop-overview] Timeloop/Accelergy Overview

类型与版本：作者项目文档；2026；访问快照 2026-11-08
支持的证据：以工作负载、体系结构、映射与技术成本模型估算性能和能量；可用周期仿真、设计数据、综合点验证。
阅读范围：Overview：分析模型、映射搜索、技术参数与验证
边界：公开，已读取所列部分
访问日期：2026-11-08
https://timeloop.csail.mit.edu/

18.24 [src-timeloop-v4] Timeloop+Accelergy v4 Architecture

类型与版本：官方格式文档；2026；v4 输入格式；访问快照 2026-11-08
支持的证据：架构包含计算/存储层次、空间实例、映射约束；显式容量、位宽、共享或分离读写带宽、缓冲份数。
阅读范围：Input Structure；spatial fanout；storage attributes
边界：公开，已读取所列部分
访问日期：2026-11-08
https://timeloop.csail.mit.edu/v4/input-formats/design/architecture

18.25 [src-mlir-transform] MLIR Transform Tutorial - Chapter 0

类型与版本：官方编译器教程；2026；滚动文档；访问快照 2026-11-08
支持的证据：结构化操作保留索引和迭代语义，有助于表达分块与融合；融合示例可去除临时缓冲。
阅读范围：Generic operations；Loop Fusion；tensor abstraction
边界：公开，已读取所列部分
访问日期：2026-11-08
https://mlir.llvm.org/docs/Tutorials/transform/Ch0/

18.26 [src-maestro] MAESTRO Documentation

类型与版本：作者工具文档；2026；所读在线文档未统一给出版本；运行需锁定 commit
支持的证据：提供数据流、性能/能量建模与结果分析入口，可作为数据复用分析工具的考察对象。
阅读范围：目录：运行、输出、支持功能、性能与能量模型
边界：公开，已读取所列部分
访问日期：2026-11-08
https://maestro.ece.gatech.edu/docs/build/html/index.html

18.27 [src-accelsim] Accel-Sim author framework repository

类型与版本：作者代码；2026；README 标识 Accel-Sim 2.0；dev 分支访问快照 2026-11-08
支持的证据：提供真实执行 trace、详细性能模型、硬件关联比较与校准组件，体现测量与模型闭环。
阅读范围：Overview；Core Components；correlator/tuner；dependencies
边界：公开，已读取所列部分
访问日期：2026-11-08
https://github.com/accel-sim/accel-sim-framework

18.28 [src-peakrdl] PeakRDL Introduction

类型与版本：官方工具文档；2026；滚动文档；SystemRDL 2.0
支持的证据：统一寄存器描述可生成 RTL、C 头文件、文档、UVM RAL 与 IP-XACT 等产物。
阅读范围：Introduction 与各输出插件简介
边界：公开，已读取所列部分
访问日期：2026-11-08
https://peakrdl.readthedocs.io/en/latest/

18.29 [src-peakrdl-limits] PeakRDL-regblock Known Limitations

类型与版本：官方限制文档；2026；访问快照 2026-11-08
支持的证据：所读版本对别名、地址对齐和 accesswidth 等有明确限制。
阅读范围：alias；unaligned registers；uniform accesswidth
边界：公开，已读取所列部分
访问日期：2026-11-08
https://peakrdl-regblock.readthedocs.io/en/latest/limitations.html

18.30 [src-orfs-autotuner] OpenROAD Flow Scripts AutoTuner

类型与版本：官方工具文档；2026；latest 访问快照 2026-11-08
支持的证据：使用参数搜索、多次流程运行及指标采集进行实现配置探索。
阅读范围：JSON 输入；sweep/tune；资源与输出指标
边界：公开，已读取所列部分
访问日期：2026-11-08
https://openroad-flow-scripts.readthedocs.io/en/latest/user/InstructionsForAutoTuner.html